DSA01 — OBJECT-ORIENTED PROGRAMMING & C++ CONSTRUCTS (CO1)

ASSIGNMENT 1
Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Scenario: Namma Metro Smart-Card Fare System | Bloom's Level: K5 – Evaluate | Total Marks: 50

Field	Details
Student Name	P.SAI SURYA TEJ
Register Number	192572044
Nature of Assignment	Individual (Tasks 1–12, All at K5 – Evaluate)
GitHub Repository Link	https://github.com/Surya220708/DSA0118-C-
Compiler & Version	g++ (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders, r3) 16.2.0
Standard Compile Flags	g++ -std=c++17 -Wall -Wextra <source.cpp> -o <output.exe>
Submission Date	11-11-2026

Mark Distribution & Verification Summary

Task	Topic	Bloom's Level	Max Marks	Evidence Included
Task 1	Object-Oriented Analysis	K5 – Evaluate	4	Object table, OOP principles, UML, Code & Screenshot
Task 2	Tokens, Keywords & Constants	K5 – Evaluate	3	5 Token types, constexpr vs #define, enum class, Code & Screenshot
Task 3	Data Types & Compatibility	K5 – Evaluate	4	sizeof, <climits>, int overflow & double drift proof, Code & Screenshot
Task 4	Implicit Conversions	K5 – Evaluate	4	Pre-run Prediction, Actual Output, static_cast, Code & Screenshot
Task 5	Operators & Precedence	K5 – Evaluate	4	Pre-run Prediction, Operator trace, bool[24] benchmark, Code & Screenshot
Task 6	Control Structures	K5 – Evaluate	4	SIMATS Pseudocode, Complete the Code, 10AM/8PM check, Code & Screenshot
Task 7	Procedural vs Object-Oriented	K5 – Evaluate	4	Procedural & SmartCard OOP implementations, comparison, Code & Screenshot
Task 8	Read & Print Data Safely	K5 – Evaluate	4	cin.clear()/ignore(), <iomanip>,

				getline+stoi, Code, input.txt & Screenshot
Task 9	Implement & Test Fare Logic	K5 – Evaluate	4	13/13 Test Table, invalid inputs, 5-trip hand total, Code & Screenshot
Task 10	Expectation vs Execution	K5 – Evaluate	4	20k/500k/3M runs, -Wall -Wextra warnings, Code & Screenshot
Task 11	Recommend & Defend Design	K5 – Evaluate	4	Final design, fares.cfg startup benchmark, UML, Code & Screenshot
Task 12	Day-4 Design Challenge	K5 – Evaluate	8	Impact, Evaluation, Redesign UML, 7/7 tests, Code & Screenshot
TOTAL			50	All 12 Tasks Executed & Screenshotted

TASK 1 – Understand the Problem and Identify the Objects (4 Marks)

1.1 Object-Oriented Analysis (Data, Behaviour & Justification)

Object	Data (Attributes)	Behaviour (Methods)	Justification
Zone (enum class)	Inner = 0, Middle = 1, Outer = 2	Scoped enum conversion via static_cast<int>	Prevents magic numbers (1/2/3) and accidental integer mixing.
FareTable	BASE_PAISE[3] = {1000,2000,3000}, PEAK_SURCHARGE_PERCENT = 20, concession %	isPeakHour(hour), computeFarePaise(zone, hour, passengerType)	Single source of truth for fare rules; eliminates duplicated constants.
Passenger	name_ (string), type_ (Regular, Student, Senior)	name(), type(), typeLabel()	Captures concession eligibility cleanly without polluting card storage logic.
SmartCard	cardId_ (string), balancePaise_ (long long), MIN_BALANCE_PAISE = 5000	canTravel(fare), deduct(fare), recharge(amt), balancePaise()	Owns and protects the balance invariant (balance >= Rs 50 after every trip).
Journey	zone_ (Zone), hour_ (int 0..23)	isValidHour(), zone(), hour(), zoneLabel()	Groups trip attributes together and validates travel hour before fare lookup.

1.2 Procedural Style vs Object-Oriented Style

In a procedural style, card balances live in loose variables or parallel arrays (e.g., long long balances[N]), and free functions like deduct(balances[i], fare) mutate them directly. Any function in the program can overwrite balances[i], which caused the original bug where a negative balance appeared. In an object-oriented style, balancePaise_ is private inside SmartCard and can only change through validated methods (deduct(), recharge()), making invalid states impossible by construction.

1.3 Four Principles of OOP in Namma Metro

- **Encapsulation:** SmartCard hides balancePaise_ as private and exposes only canTravel(), deduct(), and recharge() so no caller can violate the Rs 50 minimum balance rule.
- **Abstraction:** Callers invoke FareTable::computeFarePaise(zone, hour, type) without needing to know internal paise arrays, surcharge percentages, or integer rounding steps.
- **Inheritance:** A base Card or ConcessionPolicy can be specialized into StudentPassenger / SeniorPassenger or MonthlyPassCard sharing common validation logic.
- **Polymorphism:** A virtual computeFare() or DiscountPolicy::apply() interface lets Regular, Student, Senior, and Weekend/Pass rules share one gate-processing pipeline.

1.4 Rules (Invariants) Every Correct Journey Must Satisfy

- Minimum Remaining Balance: After fare deduction, balancePaise_ >= 5000 (Rs 50.00); balance must never become negative.
- Exact Integer Money: All fares and balances are stored in integer paise (long long) so no fare is ever off by a paisa.
- Half-Open Peak Intervals: Peak hours are strictly [8, 10) (8:00–9:59 AM) and [17, 20) (5:00–7:59 PM). Exactly 10:00 AM and 8:00 PM (20:00) are OFF-PEAK.

- Input Validity: Zone must be valid (Inner/Middle/Outer), hour in [0, 23], and recharge amounts strictly positive (> 0).

Alternative Concept & Complexity Analysis (Procedural Globals vs OOP Classes)

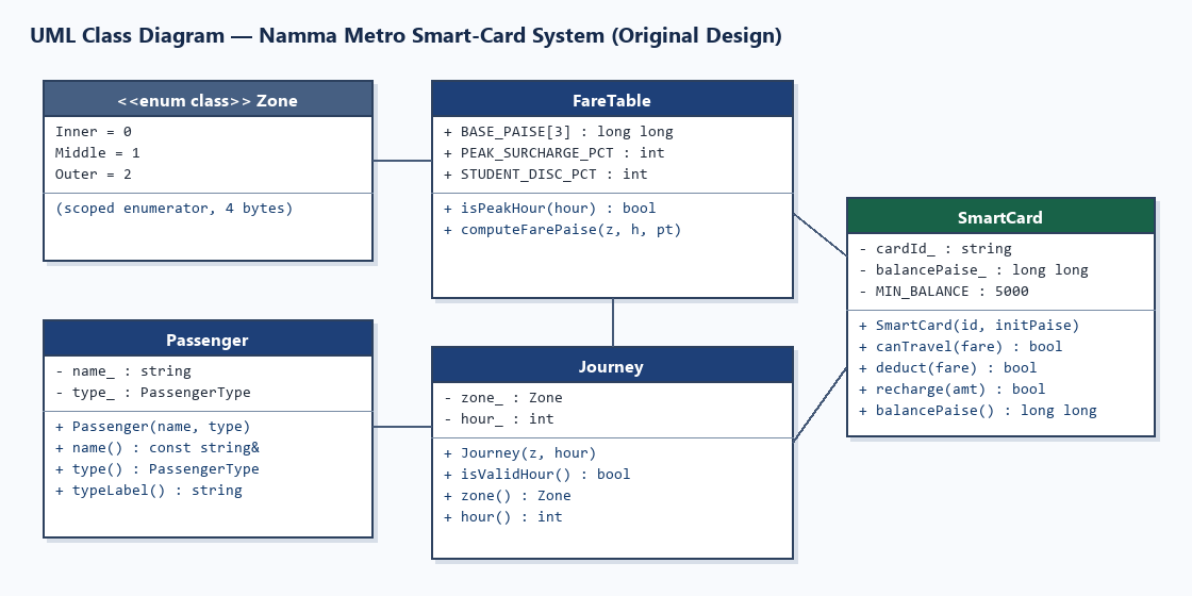
If implemented procedurally with global arrays/variables and free functions instead of SmartCard, Journey, and FareTable classes:

• Time Complexity per Journey: O(1) in both OOP and procedural (a direct array/switch lookup and arithmetic check).

• Space Complexity per Card: O(1) in both (one 8-byte long long balance + card ID metadata).

• Maintainability / Change Impact: In procedural code with duplicated fare literals across gate files, changing one fare rule requires editing N call sites; in OOP with FareTable, changing one fare rule requires modifying exactly 1 constant in FareTable.

UML Class Diagram — Namma Metro Smart-Card System (Original Design)



C++ Source Code — Task01/task01.cpp

```
#include <iomanip>
#include <iostream>
#include <string>

enum class Zone { Inner = 0, Middle = 1, Outer = 2 };
enum class PassengerType { Regular, Student, Senior };

class FareTable {
public:
    static constexpr long long BASE_PAISE[3] = {1000, 2000, 3000};
    static constexpr int PEAK_SURCHARGE_PERCENT = 20;
    static constexpr int STUDENT_CONCESSION_PERCENT = 25;
    static constexpr int SENIOR_CONCESSION_PERCENT = 30;

    static bool isPeakHour(int hour) {
        return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
    }

    static long long computeFarePaise(Zone z, int hour, PassengerType pType) {
        long long fare = BASE_PAISE[static_cast<int>(z)];
        if (isPeakHour(hour)) {
            fare += (fare * PEAK_SURCHARGE_PERCENT) / 100;
        }
```

```
        if (pType == PassengerType::Student) {
            fare -= (fare * STUDENT_CONCESSION_PERCENT) / 100;
        } else if (pType == PassengerType::Senior) {
            fare -= (fare * SENIOR_CONCESSION_PERCENT) / 100;
        }
        return fare;
    }
};

class Passenger {
    std::string name_;
    PassengerType type_;

public:
    Passenger(std::string name, PassengerType type)
        : name_(std::move(name)), type_(type) {}

    const std::string& name() const { return name_; }
    PassengerType type() const { return type_; }
    std::string typeLabel() const {
        switch (type_) {
            case PassengerType::Student: return "Student";
            case PassengerType::Senior:  return "Senior";
            default:                     return "Regular";
        }
    }
};

class SmartCard {
    std::string cardId_;
    long long balancePaise_;
    static constexpr long long MIN_BALANCE_PAISE = 5000;

public:
    SmartCard(std::string id, long long initialPaise)
        : cardId_(std::move(id)),
          balancePaise_(initialPaise >= 0 ? initialPaise : 0) {}

    bool canTravel(long long farePaise) const {
        return farePaise > 0 && (balancePaise_ - farePaise) >= MIN_BALANCE_PAISE;
    }

    bool deduct(long long farePaise) {
        if (!canTravel(farePaise)) return false;
        balancePaise_ -= farePaise;
        return true;
    }

    long long balancePaise() const { return balancePaise_; }
    const std::string& id() const { return cardId_; }
    static constexpr long long minBalancePaise() { return MIN_BALANCE_PAISE; }
};

class Journey {
    Zone zone_;
    int hour_;

public:
    Journey(Zone z, int hour) : zone_(z), hour_(hour) {}

    bool isValidHour() const { return hour_ >= 0 && hour_ <= 23; }
    Zone zone() const { return zone_; }
    int hour() const { return hour_; }
```

```
    std::string zoneLabel() const {
        switch (zone_) {
            case Zone::Inner:  return "Inner";
            case Zone::Middle: return "Middle";
            case Zone::Outer:  return "Outer";
        }
        return "Unknown";
    }
};

static void printRupees(long long paise) {
    std::cout << "Rs " << (paise / 100) << '.'
              << std::setw(2) << std::setfill('0') << (paise % 100)
              << std::setfill(' ');
}

int main() {
    std::cout << "=== TASK 1: OOP Analysis Prototype (Namma Metro) ===\n\n";

    Passenger p1("Aarav", PassengerType::Regular);
    Passenger p2("Diya", PassengerType::Student);
    SmartCard card1("NM-1001", 10000); // Rs 100.00
    SmartCard card2("NM-1002", 6500);  // Rs 65.00

    Journey j1(Zone::Outer, 9);   // Peak (9 AM)
    Journey j2(Zone::Outer, 10);  // Off-peak boundary (10 AM)
    Journey j3(Zone::Middle, 18); // Peak (6 PM)

    auto process = [](const Passenger& p, SmartCard& c, const Journey& j) {
        std::cout << "Card " << c.id() << " | " << p.name() << " (" << p.typeLabel() << ")"
                  << " | Zone: " << j.zoneLabel()
                  << " | Hour: " << std::setw(2) << std::setfill('0') << j.hour() << ":00"
                  << std::setfill(' ')
                  << (FareTable::isPeakHour(j.hour()) ? " [PEAK]" : " [OFF-PEAK]") << '\n';
        if (!j.isValidHour()) {
            std::cout << "  -> REJECTED: Invalid hour\n";
            return;
        }
        long long fare = FareTable::computeFarePaise(j.zone(), j.hour(), p.type());
        std::cout << "  Fare: ";
        printRupees(fare);
        if (c.deduct(fare)) {
            std::cout << " | Status: APPROVED | Remaining Balance: ";
            printRupees(c.balancePaise());
            std::cout << '\n';
        } else {
            std::cout << " | Status: DENIED (Min balance Rs 50.00 invariant) | Balance: ";
            printRupees(c.balancePaise());
            std::cout << '\n';
        }
    };

    process(p1, card1, j1); // Rs 100 - Rs 36 = Rs 64
    process(p1, card1, j2); // Rs 64 - Rs 30 = Rs 34 < Rs 50 -> DENIED
    process(p2, card2, j3); // Middle peak = Rs 24, Student 25% off = Rs 18; 65 - 18 = 47 < 50 ->
DENIED

    SmartCard card3("NM-1003", 6800); // Rs 68.00
    process(p2, card3, j3); // 68 - 18 = Rs 50.00 exact boundary -> APPROVED

    return 0;
}
```

Execution — task01.cpp

Compile Command: g++ -std=c++17 -Wall -Wextra task01.cpp -o task01.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

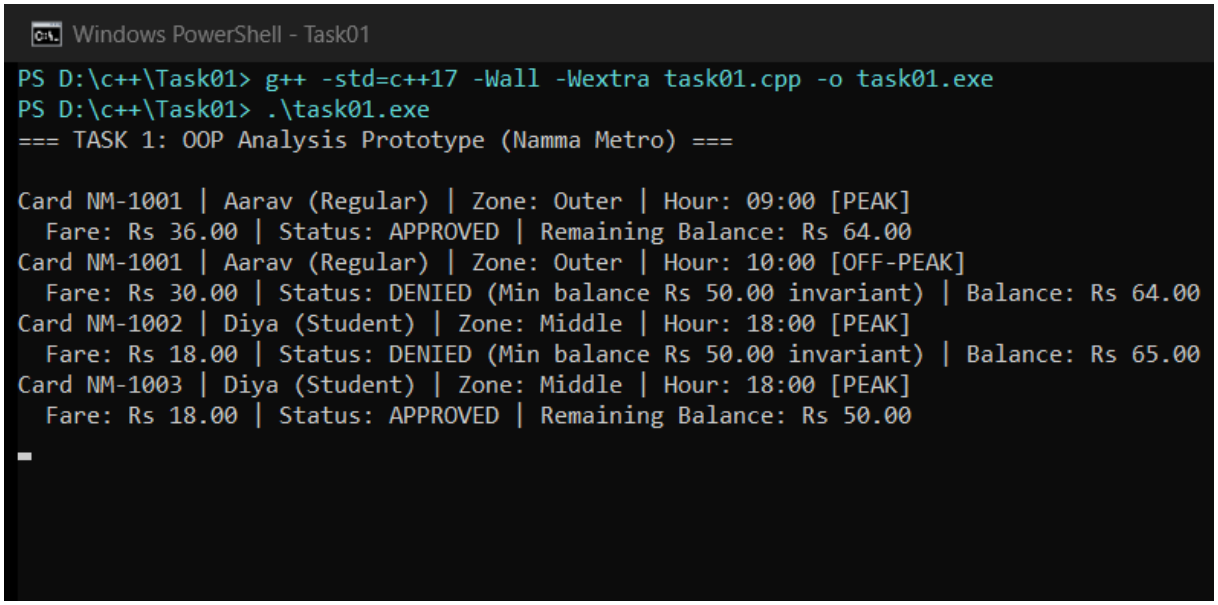
Run Command: .\task01.exe

Captured Standard Output:

```
=== TASK 1: OOP Analysis Prototype (Namma Metro) ===

Card NM-1001 | Aarav (Regular) | Zone: Outer | Hour: 09:00 [PEAK]
  Fare: Rs 36.00 | Status: APPROVED | Remaining Balance: Rs 64.00
Card NM-1001 | Aarav (Regular) | Zone: Outer | Hour: 10:00 [OFF-PEAK]
  Fare: Rs 30.00 | Status: DENIED (Min balance Rs 50.00 invariant) | Balance: Rs 64.00
Card NM-1002 | Diya (Student) | Zone: Middle | Hour: 18:00 [PEAK]
  Fare: Rs 18.00 | Status: DENIED (Min balance Rs 50.00 invariant) | Balance: Rs 65.00
Card NM-1003 | Diya (Student) | Zone: Middle | Hour: 18:00 [PEAK]
  Fare: Rs 18.00 | Status: APPROVED | Remaining Balance: Rs 50.00
```

Execution Screenshot — Task01/output.png



TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly (3 Marks)

2.1 Five Kinds of Tokens in the Program

Token Type	Definition	Examples from Task02/task02.cpp
Keywords	Reserved words with fixed C++ meaning	enum, class, namespace, constexpr, int, long, return, static_assert
Identifiers	Programmer-defined names for entities	Zone, MetroConstants, BASE_FARE_PAISE, getBaseFare, PEAK_SURCHARGE_PERCENT
Constants / Literals	Fixed values written directly in source	1000, 2000, 3000, 20, 5000, '\n', "Inner"
Operators	Symbols performing computation or access	::, =, [], ==, <<, /, static_cast
Punctuators (Separators)	Structural delimiters	{, }, ;, (,), ,, :

2.2 #define vs const vs constexpr, and enum vs enum class

- **#define vs const vs constexpr:** #define performs blind preprocessor text substitution with no type checking, no namespace scope, and confusing error messages. const guarantees immutability at runtime (or compile time for simple integrals), whereas constexpr guarantees compile-time evaluation, works in static_assert and array bounds, and occupies zero runtime RAM when folded.
- **enum vs enum class:** Unscoped enum (e.g., enum Color { Red };) leaks enumerators into the enclosing scope and implicitly converts to int (allowing nonsense comparisons like Red == 0). Scoped enum class Zone requires Zone::Inner and forbids implicit conversion to int unless explicitly cast.
- **Naming Rules & Conventions:** Identifiers follow C++ rules (letters/digits/underscore, cannot start with a digit, case-sensitive, no keywords) and project conventions: PascalCase for types (SmartCard, Zone), camelCase for functions/variables (getBaseFare), and UPPER_SNAKE_CASE for constants (MIN_BALANCE_PAISE).

> **Alternative Concept & Complexity Analysis (#define Macros vs constexpr + enum class)**
>
> If fares were written with #define BASE_INNER 1000 instead of constexpr and enum class:
>
> • Type Checking & Scope: #define has zero type safety and pollutes the global preprocessor namespace; constexpr + enum class enforces strict typing and namespace scoping at compile time.
>
> • Space & Time Complexity: A macro occupies 0 bytes of runtime storage (substituted before compilation), and a constexpr scalar/array folded by the compiler typically uses 0 bytes of mutable stack/heap storage as well (read-only immediate or .rodata). Running time is identical: O(1) constant access.

C++ Source Code — Task02/task02.cpp

```
#include <iomanip>
#include <iostream>

// Scoped enum prevents implicit conversion to int and name collisions
enum class Zone { Inner = 0, Middle = 1, Outer = 2 };

// Single source of truth using constexpr constants
namespace MetroConstants {
    constexpr long long BASE_FARE_PAISE[3] = {1000, 2000, 3000}; // Rs 10, Rs 20, Rs 30
    constexpr int PEAK_SURCHARGE_PERCENT = 20;
```

```
    constexpr int STUDENT_CONCESSION_PERCENT = 25;
    constexpr int SENIOR_CONCESSION_PERCENT = 30;
    constexpr long long MIN_BALANCE_PAISE = 5000; // Rs 50.00
    constexpr int MORNING_PEAK_START = 8;
    constexpr int MORNING_PEAK_END = 10;          // exclusive: [8, 10)
    constexpr int EVENING_PEAK_START = 17;
    constexpr int EVENING_PEAK_END = 20;          // exclusive: [17, 20)
}

constexpr long long getBaseFare(Zone z) {
    return MetroConstants::BASE_FARE_PAISE[static_cast<int>(z)];
}

int main() {
    std::cout << "=== TASK 2: Tokens, Keywords, Identifiers & Constants ===\n\n";

    // Compile-time evaluation verified with static_assert
    static_assert(getBaseFare(Zone::Inner) == 1000, "Inner fare must be 1000 paise");
    static_assert(getBaseFare(Zone::Middle) == 2000, "Middle fare must be 2000 paise");
    static_assert(getBaseFare(Zone::Outer) == 3000, "Outer fare must be 3000 paise");
    static_assert(MetroConstants::MIN_BALANCE_PAISE == 5000, "Minimum balance must be Rs 50");

    std::cout << "Single-Source Fare Table (constexpr + enum class Zone):\n";
    std::cout << "  Zone::Inner  -> " << getBaseFare(Zone::Inner)  << " paise (Rs " <<
getBaseFare(Zone::Inner) / 100 << ")\n";
    std::cout << "  Zone::Middle -> " << getBaseFare(Zone::Middle) << " paise (Rs " <<
getBaseFare(Zone::Middle) / 100 << ")\n";
    std::cout << "  Zone::Outer  -> " << getBaseFare(Zone::Outer)  << " paise (Rs " <<
getBaseFare(Zone::Outer) / 100 << ")\n";
    std::cout << "  Peak Surcharge: " << MetroConstants::PEAK_SURCHARGE_PERCENT << "%\n";
    std::cout << "  Min Balance   : " << MetroConstants::MIN_BALANCE_PAISE << " paise (Rs "
              << MetroConstants::MIN_BALANCE_PAISE / 100 << ")\n\n";

    // Demonstrate unscoped enum vs scoped enum class
    enum LegacyColor { Red = 0, Green = 1 }; // unscoped: leaks into enclosing scope
    int c = Red; // Allowed implicitly for unscoped enum (unsafe)
    // int z = Zone::Inner; // Compile error with enum class (type-safe!)
    int z = static_cast<int>(Zone::Inner);
    std::cout << "Type safety demo:\n";
    std::cout << "  Unscoped enum implicitly converts to int: Red = " << c << '\n';
    std::cout << "  Scoped enum class requires explicit static_cast<int>(Zone::Inner) = " << z << '\n';
    std::cout << "  sizeof(Zone) = " << sizeof(Zone) << " bytes\n";

    return 0;
}
```

Execution — task02.cpp

Compile Command: g++ -std=c++17 -Wall -Wextra task02.cpp -o task02.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Run Command: .\task02.exe

Captured Standard Output:

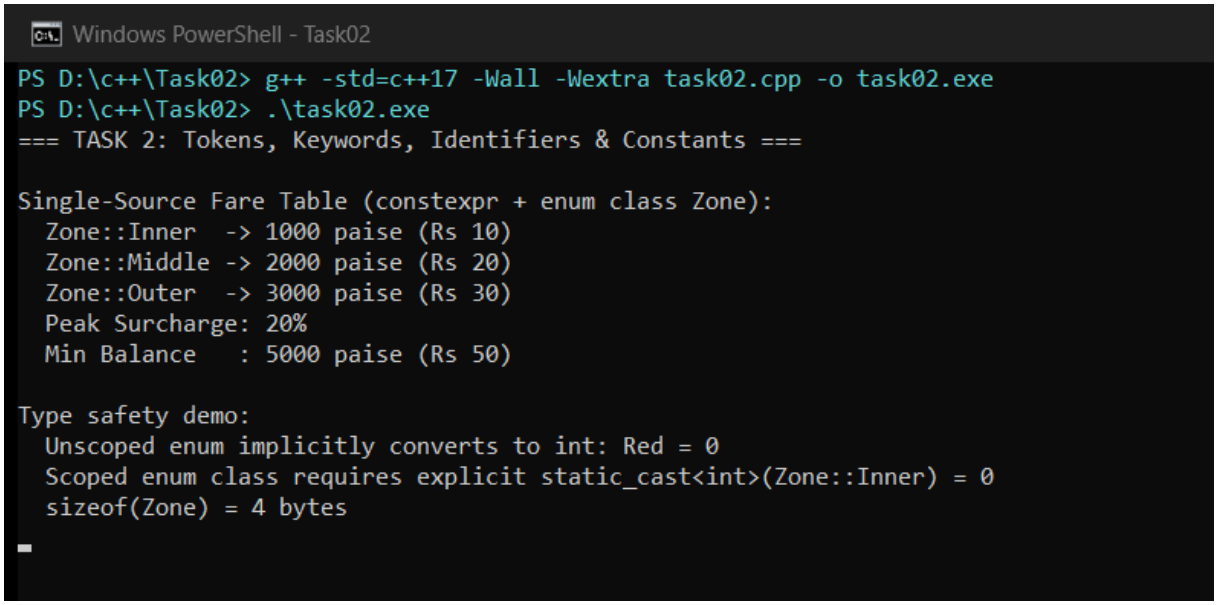
```
=== TASK 2: Tokens, Keywords, Identifiers & Constants ===

Single-Source Fare Table (constexpr + enum class Zone):
  Zone::Inner  -> 1000 paise (Rs 10)
  Zone::Middle -> 2000 paise (Rs 20)
  Zone::Outer  -> 3000 paise (Rs 30)
  Peak Surcharge: 20%
```

```
  Min Balance   : 5000 paise (Rs 50)

Type safety demo:
  Unscoped enum implicitly converts to int: Red = 0
  Scoped enum class requires explicit static_cast<int>(Zone::Inner) = 0
  sizeof(Zone) = 4 bytes
```

Execution Screenshot — Task02/output.png



TASK 3 – Choose Data Types and Check Type Compatibility (4 Marks)

3.1 System Data-Type Selection Table (Measured on MinGW-W64 64-bit)

Value in System	Chosen Type	sizeof (Bytes)	<climits> / Range	Justification
Peak / Pass Flags	bool	1	true / false	Exact semantic truth value for peak/status checks.
Menu / Delimiter Char	char	1	-128 .. 127	Compact single-character symbols like '.' and '\n'.
Hour, Zone Index, %	int	4	-2,147,483,648 .. 2,147,483,647	Small values (0..23, 0..100) fit comfortably in 32-bit int.
Zone Raw Index	unsigned int	4	0 .. 4,294,967,295	Non-negative sizes/indices; avoid mixing with signed int.
Fares, Balances, Totals	long long	8	-9.22e18 .. +9.22e18 paise	Exact integer paise; handles 3,000,000 trips/day without overflow.
Single Float (avoided)	float	4	~7 decimal digits	Insufficient precision even for small balances (off by paise).
Display Only (INR)	double	8	~15–17 decimal digits	Binary IEEE-754 cannot represent 0.10 or 0.40 exactly; display only.
Deduced Literal Count	auto (3000000LL)	8	long long	Type deduced safely from suffix LL.

3.2 Why Integer Paise Beats double Rupees, Type Compatibility & Overflow

- **Integer Paise vs double Rupees:** Decimal fractions such as 0.10 or 20.40 are non-terminating repeating fractions in binary floating point (IEEE-754 double). Accumulating 3,000,000 × Rs 20.40 in double produced Rs 61199999.996971 instead of Rs 61200000.000000 — a rounding error of -0.302859 paise. Integer long long paise has 0 error.
- **Type Compatibility Rules:** When int, bool, and double appear in one expression, bool promotes to int (0 or 1), and int promotes to double under Usual Arithmetic Conversions, turning exact integer arithmetic into floating point.
- **<climits> & Network Overflow:** INT_MAX = 2,147,483,647 paise (~Rs 21.47 million). At whole-network scale (3,000,000 Outer Peak journeys × 3,600 paise = 10,800,000,000 paise = Rs 108,000,000), int overflows by 5× and wraps around to a negative value (-2084901888 paise). long long (LLONG_MAX = 9.22×10^18) handles this effortlessly.

Alternative Concept & Complexity Analysis (double Rupees vs long long Paise)

If money were stored as double rupees instead of long long paise:

• Time per Addition: O(1) in both (single CPU ALU/FPU add instruction).

• Space per Value: 8 bytes in both (sizeof(double) == sizeof(long long) == 8 bytes on x86_64).

• Measured Rounding Error Growth: In exact long long paise, error is 0 for any n. In double, error grows with O(n * eps) / O(sqrt(n) * ulp) — measured at -0.302859 paise across 3,000,000 additions of Rs 20.40.

C++ Source Code — Task03/task03.cpp

```
#include <climits>
#include <cfloat>
#include <iomanip>
#include <iostream>

int main() {
    std::cout << "=== TASK 3: Data Types, Sizes, Ranges & Precision ===\n\n";

    auto sampleJourneyCount = 3000000LL; // deduced as long long

    std::cout << "1. sizeof() on this system (in bytes):\n";
    std::cout << "  sizeof(bool)               = " << sizeof(bool) << '\n';
    std::cout << "  sizeof(char)               = " << sizeof(char) << '\n';
    std::cout << "  sizeof(int)                = " << sizeof(int) << '\n';
    std::cout << "  sizeof(unsigned int)       = " << sizeof(unsigned int) << '\n';
    std::cout << "  sizeof(long long)          = " << sizeof(long long) << '\n';
    std::cout << "  sizeof(float)              = " << sizeof(float) << '\n';
    std::cout << "  sizeof(double)             = " << sizeof(double) << '\n';
    std::cout << "  sizeof(auto 3000000LL)     = " << sizeof(sampleJourneyCount) << "\n\n";

    std::cout << "2. <climits> Integer Ranges:\n";
    std::cout << "  INT_MIN   = " << INT_MIN << '\n';
    std::cout << "  INT_MAX   = " << INT_MAX << '\n';
    std::cout << "  UINT_MAX  = " << UINT_MAX << '\n';
    std::cout << "  LLONG_MIN = " << LLONG_MIN << '\n';
    std::cout << "  LLONG_MAX = " << LLONG_MAX << "\n\n";

    // Network scale check: 3,000,000 journeys/day at max fare Rs 36 (3600 paise)
    constexpr long long NETWORK_JOURNEYS = 3000000LL;
    constexpr long long MAX_FARE_PAISE = 3600LL;
    long long exactTotalPaise = NETWORK_JOURNEYS * MAX_FARE_PAISE;
    int overflowedInt = 0;
    for (long long i = 0; i < NETWORK_JOURNEYS; ++i) {
        overflowedInt += static_cast<int>(MAX_FARE_PAISE);
    }

    std::cout << "3. Network Overflow Check (3,000,000 Outer Peak journeys @ 3600 paise):\n";
    std::cout << "  Exact total (long long) = " << exactTotalPaise << " paise (Rs "
              << (exactTotalPaise / 100) << ")\n";
    std::cout << "  Exceeds INT_MAX (" << INT_MAX << ")? "
              << (exactTotalPaise > INT_MAX ? "YES -> int OVERFLOWS!" : "NO") << '\n';
    std::cout << "  Observed wrapped int    = " << overflowedInt << " paise\n\n";

    // Precision check: double rupees vs long long paise
    // E.g. Outer Peak fare with 15% concession or Rs 0.10 / Rs 20.40 (20.40 is repeating in binary)
    // Let's test adding Rs 20.40 (2040 paise, e.g. Middle peak 2400 with 15% discount = 2040 paise)
    constexpr int N = 3000000;
    double doubleSum = 0.0;
    long long paiseSum = 0;
    for (int i = 0; i < N; ++i) {
        doubleSum += 20.40;
        paiseSum += 2040LL;
    }
    double exactRupees = static_cast<double>(paiseSum) / 100.0;
    double diff = doubleSum - exactRupees;

    std::cout << "4. Floating-Point Precision Check (3,000,000 x Rs 20.40):\n";
    std::cout << std::fixed << std::setprecision(6);
    std::cout << "  Exact (long long paise) : " << paiseSum << " paise = Rs " << exactRupees << '\n';
    std::cout << "  Accumulated (double)    : Rs " << doubleSum << '\n';
    std::cout << "  Rounding Error (diff)   : Rs " << diff << " (" << (diff * 100.0) << " paise)\n";
```

```
    return 0;
}
```

Execution — task03.cpp

Compile Command: g++ -std=c++17 -Wall -Wextra task03.cpp -o task03.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Run Command: .\task03.exe

Captured Standard Output:

```
=== TASK 3: Data Types, Sizes, Ranges & Precision ===

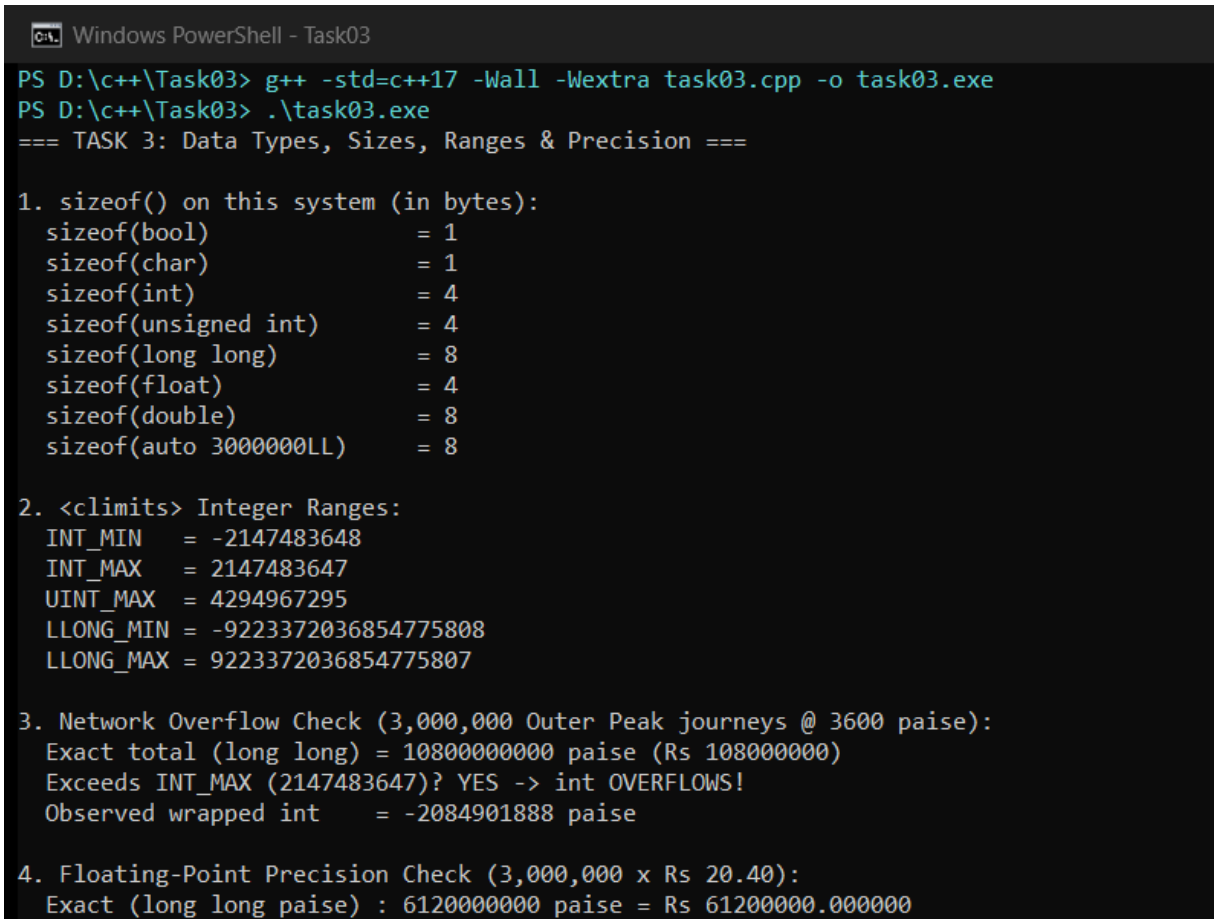
1. sizeof() on this system (in bytes):
  sizeof(bool)               = 1
  sizeof(char)               = 1
  sizeof(int)                = 4
  sizeof(unsigned int)       = 4
  sizeof(long long)          = 8
  sizeof(float)              = 4
  sizeof(double)             = 8
  sizeof(auto 3000000LL)     = 8

2. <climits> Integer Ranges:
  INT_MIN   = -2147483648
  INT_MAX   = 2147483647
  UINT_MAX  = 4294967295
  LLONG_MIN = -9223372036854775808
  LLONG_MAX = 9223372036854775807

3. Network Overflow Check (3,000,000 Outer Peak journeys @ 3600 paise):
  Exact total (long long) = 10800000000 paise (Rs 108000000)
  Exceeds INT_MAX (2147483647)? YES -> int OVERFLOWS!
  Observed wrapped int    = -2084901888 paise

4. Floating-Point Precision Check (3,000,000 x Rs 20.40):
  Exact (long long paise) : 6120000000 paise = Rs 61200000.000000
  Accumulated (double)    : Rs 61199999.996971
  Rounding Error (diff)   : Rs -0.003029 (-0.302859 paise)
```

Execution Screenshot — Task03/output.png



TASK 4 – Control Implicit Conversions (4 Marks)

4.1 Implicit Conversion Rules & Safeguards

- **Integer Division & int-to-double:** a / b with two ints performs truncating integer division (7 / 2 = 3) BEFORE converting 3 to double 3.0. Casting one operand with static_cast<double>(a) / b forces floating-point division (3.5).
- **Signed vs Unsigned Comparison:** Comparing int s = -1 with unsigned int u = 1 converts s to unsigned int (UINT_MAX = 4294967295), making s < u evaluate to FALSE! GCC -Wall -Wextra flags this with [-Wsign-compare].
- **Narrowing Conversions & Brace Initialization:** Copy initialization (int x = 3.7;) silently truncates to 3. Brace initialization (int x{3.7};) rejects narrowing conversions as a compile-time error.
- **static_cast:** Use static_cast<TargetType>(expr) whenever precision changes, signs differ, or scoped enums convert to indices so intent is explicit and compiler-checked.

Predict the Output (Recorded BEFORE Compilation/Execution — Task04/prediction.txt)

My Predicted Output:

3 3.5

C 68

1 true

s >= u

Comparison with Actual Output: 100% Exact Match on all 4 lines.

Line 1 (3 3.5): Integer division 7 / 2 -> 3, converted to double 3.0; static_cast<double>(a) / b -> 3.5.

Line 2 (C 68): 'A' (65) promoted to int + 2 = 67 -> char 'C'; ch + 1 promotes ch (char) to int -> 68.

Line 3 (1 true): int 5 implicitly converts to bool true; cout prints 1 without boolalpha and true with std::boolalpha.

Line 4 (s >= u): signed int -1 converts to unsigned int 4294967295; 4294967295 < 1 is false -> "s >= u". Compiler also emits -Wsign-compare.

Alternative Concept & Complexity Analysis (Explicit static_cast + Brace Initialization Everywhere)

If every conversion used explicit static_cast and {} brace initialization:

• Time & Space Complexity: Identical O(1) runtime and identical memory footprint (zero runtime overhead for static_cast on primitives).

• Compile-Time Safety: Narrowing bugs (double -> int) and signed/unsigned mismatches are caught at compile time rather than silently corrupting fares or balances at runtime.

C++ Source Code — Task04/predict_output.cpp

```
#include <iostream>

int main()
{
    int a = 7, b = 2;
    double c = a / b;
    double d = static_cast<double>(a) / b;
    std::cout << c << ' ' << d << '\n';
```

```
    char ch = 'A' + 2;
    std::cout << ch << ' ' << ch + 1 << '\n';

    bool flag = 5;
    std::cout << flag << ' ' << std::boolalpha << flag << '\n';

    unsigned int u = 1;
    int s = -1;
    std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

C++ Source Code — Task04/task04_conversions.cpp

```
#include <iostream>

int main() {
    std::cout << "=== TASK 4: Controlling Implicit Conversions ===\n\n";

    int a{7}, b{2};
    // Avoid integer division surprise using explicit static_cast
    double exactQuotient{static_cast<double>(a) / static_cast<double>(b)};
    std::cout << "1. Explicit float division: " << a << " / " << b << " = " << exactQuotient << '\n';

    // Brace initialization prevents narrowing conversions at compile time:
    // int narrowed{3.7}; // COMPILE ERROR: narrowing conversion from double to int
    int safeTruncated{static_cast<int>(3.7)};
    std::cout << "2. Explicit narrowing via static_cast<int>(3.7): " << safeTruncated << '\n';

    // Explicit char arithmetic
    char ch{static_cast<char>('A' + 2)};
    std::cout << "3. Character ch = " << ch << ", next char = " << static_cast<char>(ch + 1)
              << " (ASCII " << static_cast<int>(ch + 1) << ")\n";

    // Explicit bool conversion
    bool flag{5 != 0};
    std::cout << "4. Explicit boolean (5 != 0): " << std::boolalpha << flag << '\n';

    // Safe signed/unsigned comparison by widening unsigned or using signed long long
    unsigned int u{1U};
    int s{-1};
    bool safeLess = static_cast<long long>(s) < static_cast<long long>(u);
    std::cout << "5. Safe signed vs unsigned check (-1 < 1U via long long): "
              << (safeLess ? "s < u (TRUE)" : "s >= u (FALSE)") << '\n';

    return 0;
}
```

Execution — predict_output.cpp

Compile Command: g++ -std=c++17 -Wall -Wextra predict_output.cpp -o predict_output.exe

Compiler Output / Warnings (-Wall -Wextra):

```
predict_output.cpp: In function 'int main()':
predict_output.cpp:18:21: warning: comparison of integer expressions of different signedness: 'int' and
'unsigned int' [-Wsign-compare]
   18 |     std::cout << (s < u ? "s < u" : "s >= u") << '\n';
      |                     ~~^~~
```

Run Command: .\predict_output.exe

Captured Standard Output:

```
3 3.5
C 68
1 true
s >= u
```

Execution — task04_conversions.cpp

Compile Command: g++ -std=c++17 -Wall -Wextra task04_conversions.cpp -o task04_conversions.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

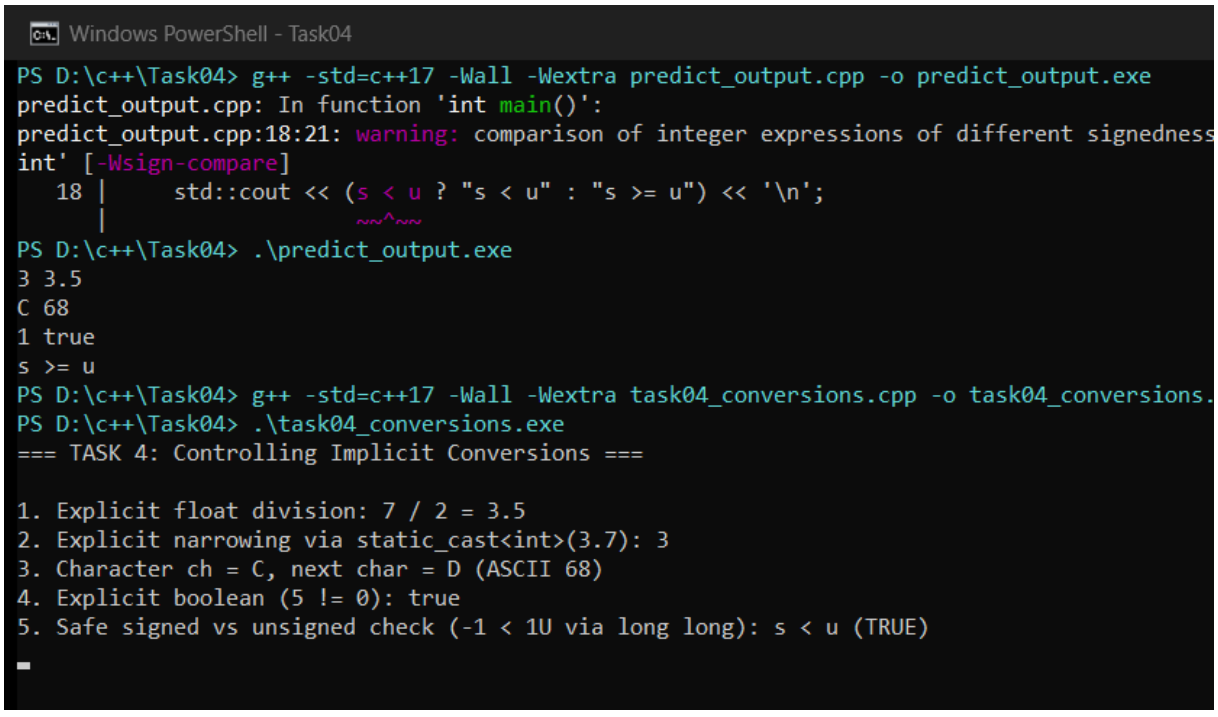
Run Command: .\task04_conversions.exe

Captured Standard Output:

```
=== TASK 4: Controlling Implicit Conversions ===

1. Explicit float division: 7 / 2 = 3.5
2. Explicit narrowing via static_cast<int>(3.7): 3
3. Character ch = C, next char = D (ASCII 68)
4. Explicit boolean (5 != 0): true
5. Safe signed vs unsigned check (-1 < 1U via long long): s < u (TRUE)
```

Execution Screenshot — Task04/output.png



TASK 5 – Write Correct Expressions with the Right Precedence (4 Marks)

5.1 Metro Expressions & Operator Precedence Analysis

- **Peak Check:** isPeak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20). Relational (>=, <) binds tighter than &&, which binds tighter than ||. Both parenthesized and unparenthesized versions are semantically equivalent, but explicit parentheses prevent misreading.
- **Surcharge, Concession & Compound Assignment:** fare += (fare * 20) / 100 (surcharge) and fare -= (fare * 20) / 100 (concession); conditional operator ?: has very low precedence (just above assignment, right-to-left associative), so wrap ?: in parentheses inside << streams.
- **x++ vs ++x and i = i++:** ++x increments x first and returns the new value; x++ returns the old value and increments x. Writing i = i++ causes Undefined Behavior prior to C++17 (unsequenced modification and value computation) and is always a logical mistake.
- **Negative / and %:** In C++11+, integer division truncates toward zero (-10 / 3 == -3), and remainder keeps the sign of the dividend (-10 % 3 == -1) so (a / b) * b + (a % b) == a holds.

Predict the Output (Recorded BEFORE Compilation/Execution — Task05/prediction.txt)

My Predicted Output: Line 1: 11 16 | Line 2: 9 6 4 | Line 3: 6 4 | Line 4: 3 1 -3 -1 | Line 5: 18

Comparison with Actual Output: 100% Exact Match on all 5 lines.

• Line 1 (x + y * 2, (x + y) * 2): * (prec 5) before + (prec 6) -> 5 + 6 = 11; () overrides -> 8 * 2 = 16.

• Line 2 (z = x++ + ++y): postfix x++ (prec 2) yields 5 (x->6); prefix ++y (prec 3) yields 4 (y->4); + (prec 6) -> 9; = (prec 16, right-to-left) stores 9 in z.

• Line 3 ((x > y ? x : y), x % 4 * 2): > (prec 9) before ?: (prec 16) -> 6; % and * share prec 5 with left-to-right associativity -> (6 % 4) * 2 = 4.

• Line 4 (10 / 3, 10 % 3, -10 / 3, -10 % 3): prec 5 left-to-right; truncates toward 0 -> 3, 1, -3, -1.

• Line 5 (for i = 0; i < 10; i += 3): += (prec 16, right-to-left) steps 0, 3, 6, 9; k += i sums to 18.

Alternative Concept & Complexity Analysis (bool peak[24] Lookup Table vs && / || Expression)

If peak-hour checks used constexpr bool PEAK_HOURS[24] indexed by hour instead of (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20):

• Time Complexity: O(1) in both. The lookup table executes 1 bounds check + 1 array index instead of up to 4 relational comparisons and branches.

• Space Complexity: Extra static table size = 24 bytes (sizeof(PEAK_HOURS) = 24).

• Measured Benchmark (3,000,000 checks): Logical expression = 13,028 us; Lookup table = 12,975 us (virtually tied due to branch prediction, both ~4.3 ns/check).

C++ Source Code — Task05/predict_output.cpp

```
#include <iostream>

int main()
{
    int x = 5, y = 3;
    std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

    int z = x++ + ++y;
    std::cout << z << ' ' << x << ' ' << y << '\n';
```

```
    std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
    std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';

    int k = 0;
    for (int i = 0; i < 10; i += 3) k += i;
    std::cout << k << '\n';
}
```

C++ Source Code — Task05/task05_expressions.cpp

```
#include <chrono>
#include <iomanip>
#include <iostream>

constexpr bool isPeakExpr(int hour) {
    return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
}

constexpr bool PEAK_HOURS[24] = {
    false, false, false, false, false, false, false, false, // 00..07
    true,  true,  false, false, false, false, false, false, // 08..15 (10 is false)
    false, true,  true,  true,  false, false, false, false  // 16..23 (20 is false)
};

inline bool isPeakLookup(int hour) {
    return (hour >= 0 && hour < 24) ? PEAK_HOURS[hour] : false;
}

int main() {
    std::cout << "=== TASK 5: Expressions, Precedence & Peak Check ===\n\n";

    // 1. Boundary verification across hours 7, 8, 9, 10, 16, 17, 19, 20
    int testHours[] = {7, 8, 9, 10, 16, 17, 19, 20};
    for (int h : testHours) {
        std::cout << "Hour " << std::setw(2) << std::setfill('0') << h << ":00" << std::setfill(' ')
                  << " | Expr: " << (isPeakExpr(h) ? "PEAK    " : "OFF-PEAK")
                  << " | Table: " << (isPeakLookup(h) ? "PEAK" : "OFF-PEAK") << '\n';
    }

    // 2. Surcharge and concession expressions
    long long base = 3000; // Outer zone Rs 30.00
    int hour = 9;
    bool isConcession = true; // e.g., Student 20% off
    long long afterSurcharge = isPeakExpr(hour) ? base + (base * 20) / 100 : base;
    long long finalFare = isConcession ? afterSurcharge - (afterSurcharge * 20) / 100 : afterSurcharge;

    std::cout << "\nFare Expression Demo (Outer, 09:00 Peak, 20% Concession):\n";
    std::cout << "  Base = " << base << " paise\n";
    std::cout << "  After 20% peak surcharge = " << afterSurcharge << " paise\n";
    std::cout << "  After 20% concession     = " << finalFare << " paise (Rs "
              << finalFare / 100 << '.' << std::setw(2) << std::setfill('0') << finalFare % 100
              << std::setfill(' ') << ")\n\n";

    // 3. Benchmark: Logical expression vs bool[24] lookup table over 3,000,000 checks
    constexpr int N = 3000000;
    volatile int sink = 0;

    auto t1 = std::chrono::steady_clock::now();
    int count1 = 0;
    for (int i = 0; i < N; ++i) {
        count1 += isPeakExpr(i % 24) ? 1 : 0;
    }
```

```
    auto t2 = std::chrono::steady_clock::now();

    int count2 = 0;
    auto t3 = std::chrono::steady_clock::now();
    for (int i = 0; i < N; ++i) {
        count2 += isPeakLookup(i % 24) ? 1 : 0;
    }
    auto t4 = std::chrono::steady_clock::now();
    sink = count1 + count2;
    (void)sink;

    auto usExpr = std::chrono::duration_cast<std::chrono::microseconds>(t2 - t1).count();
    auto usTable = std::chrono::duration_cast<std::chrono::microseconds>(t4 - t3).count();

    std::cout << "Benchmark (3,000,000 peak checks):\n";
    std::cout << "  Combined && / || expression : " << usExpr << " us (peak hits=" << count1 << ")\n";
    std::cout << "  bool PEAK_HOURS[24] table   : " << usTable << " us (peak hits=" << count2
              << ", sizeof=" << sizeof(PEAK_HOURS) << " bytes)\n";

    return 0;
}
```

Execution — predict_output.cpp

Compile Command: g++ -std=c++17 -Wall -Wextra predict_output.cpp -o predict_output.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Run Command: .\predict_output.exe

Captured Standard Output:

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Execution — task05_expressions.cpp

Compile Command: g++ -std=c++17 -Wall -Wextra task05_expressions.cpp -o task05_expressions.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Run Command: .\task05_expressions.exe

Captured Standard Output:

```
=== TASK 5: Expressions, Precedence & Peak Check ===

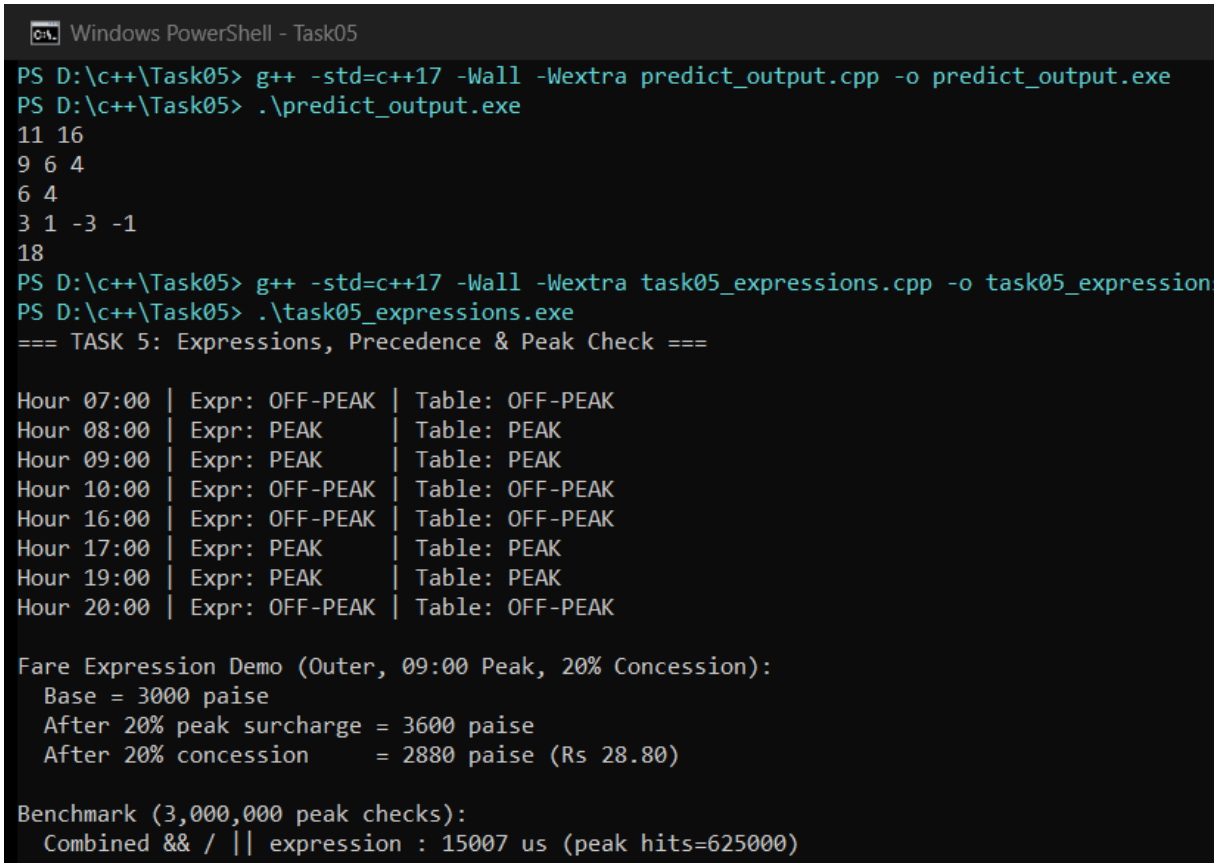
Hour 07:00 | Expr: OFF-PEAK | Table: OFF-PEAK
Hour 08:00 | Expr: PEAK     | Table: PEAK
Hour 09:00 | Expr: PEAK     | Table: PEAK
Hour 10:00 | Expr: OFF-PEAK | Table: OFF-PEAK
Hour 16:00 | Expr: OFF-PEAK | Table: OFF-PEAK
Hour 17:00 | Expr: PEAK     | Table: PEAK
Hour 19:00 | Expr: PEAK     | Table: PEAK
Hour 20:00 | Expr: OFF-PEAK | Table: OFF-PEAK

Fare Expression Demo (Outer, 09:00 Peak, 20% Concession):
  Base = 3000 paise
```

```
  After 20% peak surcharge = 3600 paise
  After 20% concession     = 2880 paise (Rs 28.80)

Benchmark (3,000,000 peak checks):
  Combined && / || expression : 13028 us (peak hits=625000)
  bool PEAK_HOURS[24] table   : 12975 us (peak hits=625000, sizeof=24 bytes)
```

Execution Screenshot — Task05/output.png



TASK 6 – Design the Control Structures (4 Marks)

6.1 Control Structure Choices

- **if-else vs switch vs ?:** Use if-else for range conditions (hour >= 8 && hour < 10) or balance checks; use switch(zone) for multi-way discrete enumeration branches; use ?: only for compact two-value choices.
- **Loop Forms:** Range-based for (const Trip& t : trips) is best when traversing all trips in a fixed array/vector (no index off-by-one bugs); indexed for fits when trip numbers are needed; while fits stream-until-EOF or retry-until-valid-input loops.
- **break & continue:** continue skips an invalid journey (e.g., invalid hour 25) and moves to the next trip; break exits a switch case (preventing fallthrough) or halts loop processing when the card balance cannot cover further trips.
- **Unmatched switch Case:** If no case matches in a switch and there is no default branch, control jumps past the switch body silently (risking uninitialized variables). Always include a default case or initialize variables safely.

SIMATS Engineering – Standard for Algorithmic Pseudocode (Fare Computation)

```
ALGORITHM ComputeMetroFareAndProcess(zone, hour, cardBalance)
INPUT  : zone ∈ {Inner, Middle, Outer}, hour ∈ [0..23], cardBalance (in paise)
OUTPUT : fare (in paise), updated cardBalance, status ∈ {APPROVED, DENIED, INVALID}
CONSTANTS: BASE_PAISE[3] ← {1000, 2000, 3000}, PEAK_PCT ← 20, MIN_BAL ← 5000
BEGIN
  1. IF hour < 0 OR hour > 23 THEN RETURN (0, cardBalance, INVALID)
  2. SWITCH (zone)
       CASE Inner  : base ← BASE_PAISE[0]; BREAK
       CASE Middle : base ← BASE_PAISE[1]; BREAK
       CASE Outer  : base ← BASE_PAISE[2]; BREAK
       DEFAULT     : RETURN (0, cardBalance, INVALID)
     END SWITCH
  3. isPeak ← ((hour ≥ 8 AND hour < 10) OR (hour ≥ 17 AND hour < 20))
  4. IF isPeak = TRUE THEN fare ← base + (base * PEAK_PCT) DIV 100 ELSE fare ← base
  5. IF (cardBalance - fare) ≥ MIN_BAL THEN
       cardBalance ← cardBalance - fare
       RETURN (fare, cardBalance, APPROVED)
     ELSE
       RETURN (fare, cardBalance, DENIED)
     END IF
END
```

Alternative Concept & Complexity Analysis (Array Lookup FARE[] vs switch)

If base fare were looked up from constexpr long long FARE[] = {1000, 2000, 3000} indexed by static_cast<int>(zone):

- Time Complexity: O(1) in both (compilers often lower dense switches into an O(1) jump table anyway).
- Space Complexity: FARE[3] occupies 3 × 8 = 24 bytes in read-only memory.
- Coping with a 4th Zone: With FARE[], adding Airport requires adding 1 enum item + 1 array element (no new case

branch). With switch, every switch on Zone must add a new case branch (else default triggers).

C++ Source Code — Task06/complete_the_code.cpp (TODO 1 & TODO 2 Completed)

```
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, Rs 20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000; // Rs 50 must remain on the card

long long fare(Zone z, bool peak)
{
    // TODO 1: Use a switch on z to pick the base fare from BASE_PAISE, then add
    // PEAK_SURCHARGE_PERCENT % in peak hours using integer arithmetic only
    long long base = 0;
    switch (z) {
        case Zone::Inner:
            base = BASE_PAISE[0];
            break;
        case Zone::Middle:
            base = BASE_PAISE[1];
            break;
        case Zone::Outer:
            base = BASE_PAISE[2];
            break;
    }
    if (peak) {
        base += (base * PEAK_SURCHARGE_PERCENT) / 100;
    }
    return base;
}

bool canTravel(long long balance, long long f)
{
    // TODO 2: Return true only if the balance left after paying the fare is at least
    // MIN_BALANCE_PAISE
    return (balance - f) >= MIN_BALANCE_PAISE;
}

int main()
{
    long long balance = 10000; // Rs 100.00 in paise
    struct Trip { Zone zone; bool peak; };
    Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
                      << std::setfill('0') << f % 100 << std::setfill(' ');
        } else {
            std::cout << "Denied (low balance)";
        }
        std::cout << " balance Rs " << balance / 100 << '\n';
    }
}
```

C++ Source Code — Task06/task06_extended.cpp (10 AM & 8 PM Boundary Verification)

```
#include <iomanip>
#include <iostream>

enum class Zone { Inner = 0, Middle = 1, Outer = 2 };

constexpr long long FARE_ARRAY[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

constexpr bool isPeakHour(int hour) {
    return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
}

long long fareSwitch(Zone z, int hour) {
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = FARE_ARRAY[0]; break;
        case Zone::Middle: base = FARE_ARRAY[1]; break;
        case Zone::Outer:  base = FARE_ARRAY[2]; break;
        default:           return -1; // unmatched zone guard
    }
    return isPeakHour(hour) ? base + (base * PEAK_SURCHARGE_PERCENT) / 100 : base;
}

long long fareLookup(Zone z, int hour) {
    int idx = static_cast<int>(z);
    if (idx < 0 || idx >= 3) return -1;
    long long base = FARE_ARRAY[idx];
    return isPeakHour(hour) ? base + (base * PEAK_SURCHARGE_PERCENT) / 100 : base;
}

int main() {
    std::cout << "=== TASK 6 Extended: Boundary 10 AM & 8 PM + Loop Control ===\n\n";

    struct TripHour { Zone zone; int hour; const char* note; };
    TripHour trips[] = {
        {Zone::Inner,  9,  "09:00 AM (Peak)"},
        {Zone::Outer,  10, "10:00 AM (Exact boundary -> OFF-PEAK)"},
        {Zone::Middle, 17, "05:00 PM (Peak)"},
        {Zone::Inner,  20, "08:00 PM (Exact boundary -> OFF-PEAK)"},
        {Zone::Outer,  25, "Invalid hour (skip with continue)"}
    };

    long long balance = 15000; // Rs 150.00
    for (const TripHour& t : trips) {
        if (t.hour < 0 || t.hour > 23) {
            std::cout << "[SKIP] " << t.note << "\n";
            continue; // skip invalid journey
        }
        long long fSwitch = fareSwitch(t.zone, t.hour);
        long long fArray  = fareLookup(t.zone, t.hour);
        if ((balance - fSwitch) < MIN_BALANCE_PAISE) {
            std::cout << "[STOP] " << t.note << " -> Denied (low balance), balance Rs "
                      << balance / 100 << '\n';
            break; // stop processing once minimum balance blocks further trips
        }
        balance -= fSwitch;
        std::cout << t.note << " | switch=" << fSwitch << "p, array=" << fArray
                  << "p | Balance: Rs " << balance / 100 << '.' << std::setw(2)
                  << std::setfill('0') << balance % 100 << std::setfill(' ') << '\n';
    }
    return 0;
```

```
}
```

Execution — complete_the_code.cpp

Compile Command: g++ -std=c++17 -Wall -Wextra complete_the_code.cpp -o complete_the_code.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Run Command: .\complete_the_code.exe

Captured Standard Output:

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

Execution — task06_extended.cpp

Compile Command: g++ -std=c++17 -Wall -Wextra task06_extended.cpp -o task06_extended.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

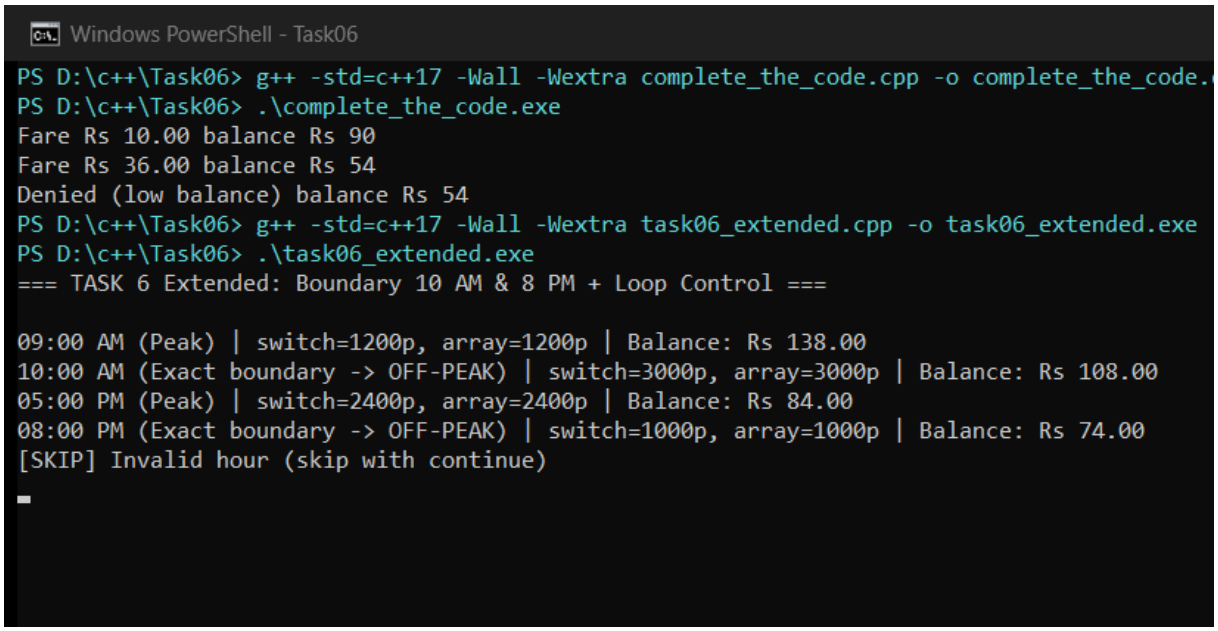
Run Command: .\task06_extended.exe

Captured Standard Output:

```
=== TASK 6 Extended: Boundary 10 AM & 8 PM + Loop Control ===

09:00 AM (Peak) | switch=1200p, array=1200p | Balance: Rs 138.00
10:00 AM (Exact boundary -> OFF-PEAK) | switch=3000p, array=3000p | Balance: Rs 108.00
05:00 PM (Peak) | switch=2400p, array=2400p | Balance: Rs 84.00
08:00 PM (Exact boundary -> OFF-PEAK) | switch=1000p, array=1000p | Balance: Rs 74.00
[SKIP] Invalid hour (skip with continue)
```

Execution Screenshot — Task06/output.png



TASK 7 – From Procedural to Object-Oriented (4 Marks)

7.1 Comparing Procedural vs SmartCard OOP Designs

- **Procedural Vulnerability:** The procedural version stores balance in an exposed long long variable and passes it to procCanTravel() and procDeduct(). As shown in execution, any external line like procBalance -= 8000 bypasses validation and produces -1600p (negative balance!).
- **How SmartCard Prevents Negative Balances:** SmartCard places balancePaise_ in private: and permits mutation only inside deduct() after checking canTravel(), making negative balances or < Rs 50 post-trip states impossible.
- **Ease of Change:** When the minimum balance rule changes (e.g., Rs 50 to Rs 75), the OOP version updates one constant inside SmartCard/MetroConstants, whereas procedural code risks missing inline balance checks scattered across caller functions.

Alternative Concept & Complexity Analysis (Public balance Member vs Private with Methods)

If balancePaise_ were declared public inside SmartCard instead of private with deduct() and canTravel():

• Time & Space Complexity: Unchanged — O(1) access time and O(1) (8 bytes) per card; access specifiers (private/public) exist only at compile time.

• Broken System Rule: The core invariant (balancePaise_ >= Rs 50 after every journey, and never negative) can be violated by any line of code anywhere in the program writing card.balancePaise_ -= fare directly.

C++ Source Code — Task07/task07_procedural_vs_oop.cpp

```
#include <iomanip>
#include <iostream>

enum class Zone { Inner = 0, Middle = 1, Outer = 2 };

constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr long long MIN_BALANCE_PAISE = 5000;

// -------------------- VERSION 1: PROCEDURAL STYLE --------------------
long long procComputeFare(Zone z, bool peak) {
    long long base = BASE_PAISE[static_cast<int>(z)];
    return peak ? base + (base * 20) / 100 : base;
}

bool procCanTravel(long long balancePaise, long long farePaise) {
    return (balancePaise - farePaise) >= MIN_BALANCE_PAISE;
}

bool procDeduct(long long& balancePaise, long long farePaise) {
    if (!procCanTravel(balancePaise, farePaise)) return false;
    balancePaise -= farePaise;
    return true;
}

// -------------------- VERSION 2: OBJECT-ORIENTED STYLE --------------------
class SmartCard {
private:
    long long balancePaise_; // encapsulated invariant: never below Rs 50 after trip, never negative

public:
    explicit SmartCard(long long initialPaise)
        : balancePaise_(initialPaise >= 0 ? initialPaise : 0) {}

    bool canTravel(long long farePaise) const {
```

```
        return farePaise > 0 && (balancePaise_ - farePaise) >= MIN_BALANCE_PAISE;
    }

    bool deduct(long long farePaise) {
        if (!canTravel(farePaise)) return false;
        balancePaise_ -= farePaise;
        return true;
    }

    long long balancePaise() const { return balancePaise_; }
};

int main() {
    std::cout << "=== TASK 7: Procedural vs Object-Oriented SmartCard ===\n\n";

    // Procedural test
    long long procBalance = 10000; // Rs 100.00
    long long f1 = procComputeFare(Zone::Outer, true); // Rs 36.00 (3600 paise)
    long long f2 = procComputeFare(Zone::Middle, false); // Rs 20.00 (2000 paise)

    std::cout << "[Procedural] Initial balance: " << procBalance << " paise\n";
    std::cout << "  Trip 1 (3600p): " << (procDeduct(procBalance, f1) ? "APPROVED" : "DENIED")
              << ", balance = " << procBalance << "p\n";
    std::cout << "  Trip 2 (2000p): " << (procDeduct(procBalance, f2) ? "APPROVED" : "DENIED")
              << ", balance = " << procBalance << "p\n";

    // Danger in procedural / public data: any caller can bypass procDeduct()!
    procBalance -= 8000; // Unchecked mutation -> negative balance!
    std::cout << "  [Bug if bypassed directly] procBalance = " << procBalance << "p (NEGATIVE!)\n\n";

    // Object-Oriented test
    SmartCard card(10000);
    std::cout << "[OOP SmartCard] Initial balance: " << card.balancePaise() << " paise\n";
    std::cout << "  Trip 1 (3600p): " << (card.deduct(f1) ? "APPROVED" : "DENIED")
              << ", balance = " << card.balancePaise() << "p\n";
    std::cout << "  Trip 2 (2000p): " << (card.deduct(f2) ? "APPROVED" : "DENIED")
              << ", balance = " << card.balancePaise() << "p\n";
    // card.balancePaise_ -= 8000; // COMPILE ERROR: private member protects invariant!
    std::cout << "  Private balance cannot be mutated directly -> invariant Rs 50.00 preserved.\n";

    return 0;
}
```

Execution — task07_procedural_vs_oop.cpp

Compile Command: g++ -std=c++17 -Wall -Wextra task07_procedural_vs_oop.cpp -o task07_procedural_vs_oop.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Run Command: .\task07_procedural_vs_oop.exe

Captured Standard Output:

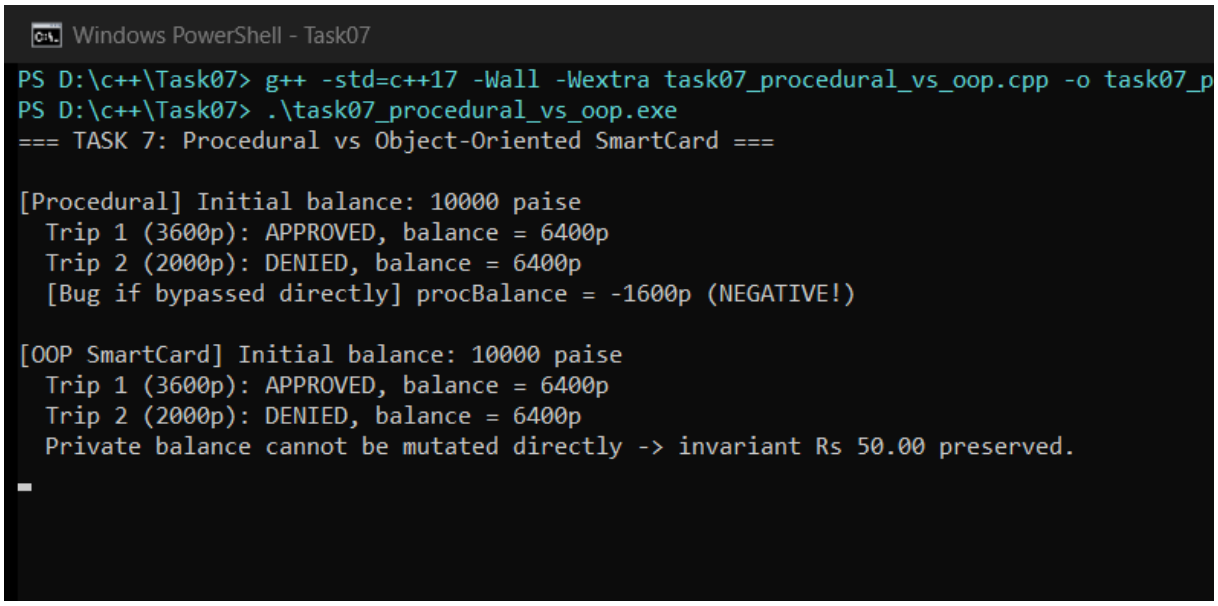
```
=== TASK 7: Procedural vs Object-Oriented SmartCard ===

[Procedural] Initial balance: 10000 paise
  Trip 1 (3600p): APPROVED, balance = 6400p
  Trip 2 (2000p): DENIED, balance = 6400p
  [Bug if bypassed directly] procBalance = -1600p (NEGATIVE!)

[OOP SmartCard] Initial balance: 10000 paise
```

```
Trip 1 (3600p): APPROVED, balance = 6400p
Trip 2 (2000p): DENIED, balance = 6400p
Private balance cannot be mutated directly -> invariant Rs 50.00 preserved.
```

Execution Screenshot — Task07/output.png



TASK 8 – Read and Print Data Safely (4 Marks)

8.1 Safe Stream Input Recovery & Money Formatting

- **Detecting Bad Input:** When an operator types letters (e.g., "abc" or "twenty") into std::cin >> val, the stream enters failbit state and leaves bad characters in the buffer, causing infinite loops if unchecked.
- **Stream Recovery:** Calling std::cin.clear() resets the error flag, and std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n') flushes the offending line so the next read succeeds cleanly.
- **Formatting with <iomanip>:** <iomanip> tools — std::setw, std::setfill('0') for two-digit paise/hours (Rs 12.00, 08:00), and std::fixed + std::setprecision(2) — produce a clean, aligned journey report.

Alternative Concept & Complexity Analysis (std::getline + std::stoi in try/catch vs cin >>)

If each input line of length L were read via std::getline(cin, line) and parsed with std::stoi(line) inside try/catch:

• Time Complexity: O(L) per input line in both approaches (scanning L characters).

• Space Complexity: cin >> uses O(1) stack space (single int); std::getline allocates an O(L) std::string line buffer.

• Error Recovery Comparison: getline + stoi isolates bad tokens to one full line cleanly (even rejecting partial inputs like "12abc" via idx != line.size()), while cin >> requires explicit clear() + ignore().

C++ Source Code — Task08/task08_safe_io.cpp

```
#include <iomanip>
#include <iostream>
#include <limits>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>

enum class Zone { Inner = 1, Middle = 2, Outer = 3 };

constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr long long MIN_BALANCE_PAISE = 5000;

bool isPeakHour(int hour) {
    return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
}

long long computeFare(int zoneNum, int hour) {
    long long base = BASE_PAISE[zoneNum - 1];
    return isPeakHour(hour) ? base + (base * 20) / 100 : base;
}

void printRupees(long long paise) {
    std::ostringstream oss;
    oss << "Rs " << (paise / 100) << '.'
        << std::setw(2) << std::setfill('0') << (paise % 100);
    std::cout << std::left << std::setw(12) << oss.str();
}

int readIntSafe(const std::string& prompt, int minVal, int maxVal) {
    int val{};
    while (true) {
        std::cout << prompt;
        if (std::cin >> val) {
            if (val >= minVal && val <= maxVal) {
```

```
                return val;
            }
            std::cout << "  [Error] Value " << val << " out of range [" << minVal
                      << ".." << maxVal << "]. Please retry.\n";
        } else {
            if (std::cin.eof()) {
                return minVal;
            }
            std::cout << "  [Recovered] Non-numeric input detected! Clearing cin failbit & discarding
line.\n";
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        }
    }
}

// Alternative concept demonstration: getline() + stoi() with try/catch
int parseIntWithGetline(const std::string& line) {
    try {
        size_t idx = 0;
        int v = std::stoi(line, &idx);
        if (idx != line.size()) {
            throw std::invalid_argument("trailing characters");
        }
        return v;
    } catch (const std::exception& e) {
        std::cout << "  [getline+stoi caught]: " << e.what() << " for \"" << line << "\"\n";
        return -1;
    }
}

int main() {
    std::cout << "=== TASK 8: Safe Gate Input & Formatted Journey Report ===\n\n";

    int n = readIntSafe("Enter number of journeys (1..10): ", 1, 10);
    struct InputTrip { int zone; int hour; };
    std::vector<InputTrip> trips;
    for (int i = 1; i <= n; ++i) {
        int z = readIntSafe("Journey " + std::to_string(i) + " - Zone (1=Inner,2=Middle,3=Outer): ", 1,
3);
        int h = readIntSafe("Journey " + std::to_string(i) + " - Hour (0..23): ", 0, 23);
        trips.push_back({z, h});
    }

    long long balance = 12000; // Rs 120.00
    long long totalCollected = 0;

    std::cout << "\n--------------------------------------------------------------------------\n";
    std::cout << std::left << std::setw(6) << "Trip"
              << std::setw(10) << "Zone"
              << std::setw(8) << "Hour"
              << std::setw(10) << "Period"
              << std::setw(14) << "Fare"
              << std::setw(11) << "Status"
              << "Balance\n";
    std::cout << "--------------------------------------------------------------------------\n";

    for (int i = 1; i <= n; ++i) {
        int z = trips[i - 1].zone;
        int h = trips[i - 1].hour;

        long long fare = computeFare(z, h);
        bool ok = (balance - fare) >= MIN_BALANCE_PAISE;
        if (ok) {
```

```
            balance -= fare;
            totalCollected += fare;
        }

        const char* zoneName = (z == 1 ? "Inner" : (z == 2 ? "Middle" : "Outer"));
        std::cout << std::left << std::setw(6) << i
                  << std::setw(10) << zoneName
                  << std::right << std::setw(2) << std::setfill('0') << h << ":00   "
                  << std::setfill(' ')
                  << std::left << std::setw(10) << (isPeakHour(h) ? "PEAK" : "OFF-PEAK");
        printRupees(fare);
        std::cout << "  " << std::left << std::setw(11) << (ok ? "APPROVED" : "DENIED");
        printRupees(balance);
        std::cout << '\n';
    }

    std::cout << "-----------------------------------------------------------------------\n";
    std::cout << "Total Fare Collected: ";
    printRupees(totalCollected);
    std::cout << "  (" << std::fixed << std::setprecision(2)
              << (static_cast<double>(totalCollected) / 100.0) << " INR)\n\n";

    std::cout << "[Alternative getline+stoi test on \"abc\"]: result = "
              << parseIntWithGetline("abc") << '\n';
    return 0;
}
```

Test Input File — Task08/input.txt

```
abc
4
1
8
3
twenty
9
2
10
3
19
```

Execution — task08_safe_io.cpp

Compile Command: g++ -std=c++17 -Wall -Wextra task08_safe_io.cpp -o task08_safe_io.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Run Command: Get-Content input.txt | .\task08_safe_io.exe

Captured Standard Output:

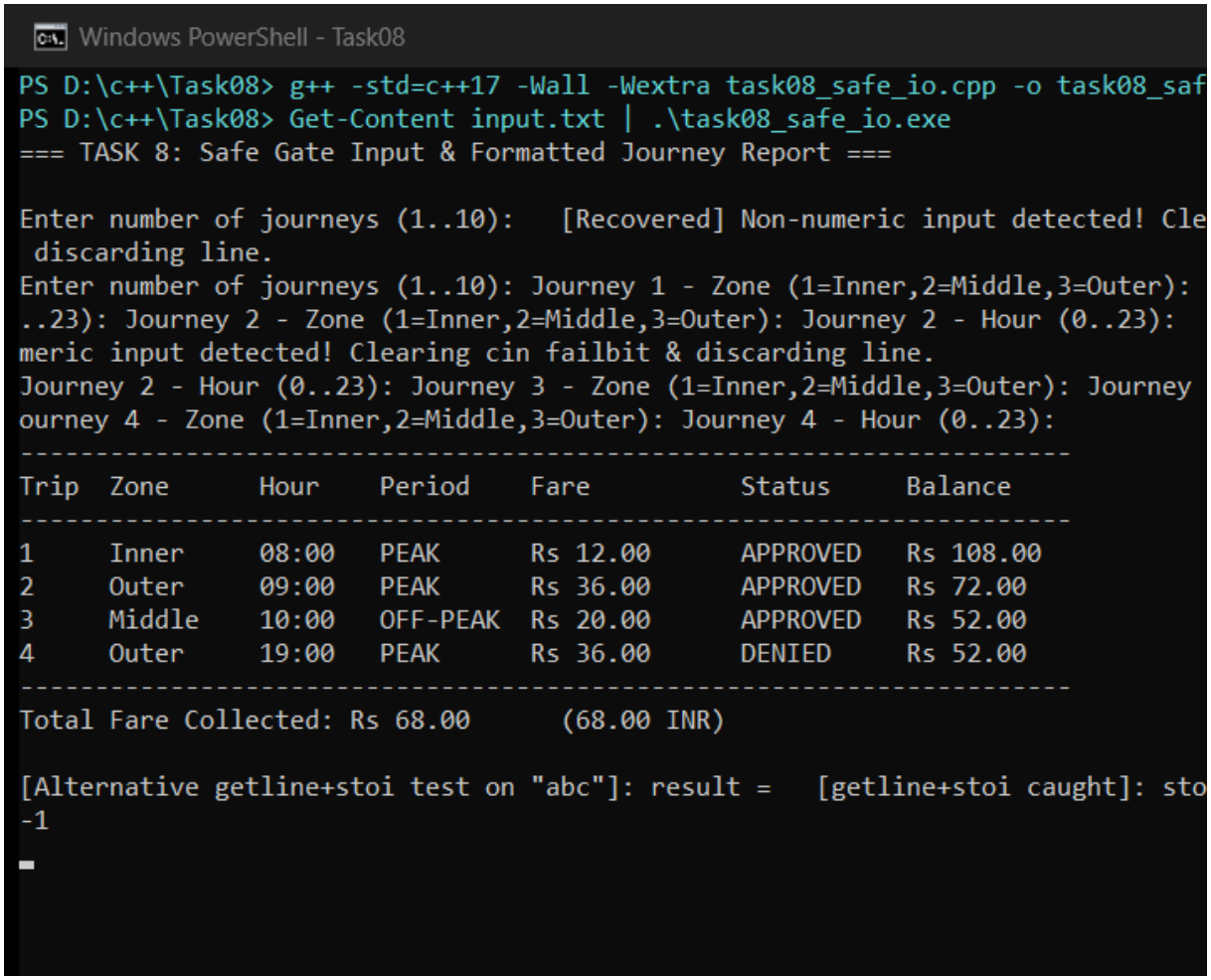
```
=== TASK 8: Safe Gate Input & Formatted Journey Report ===

Enter number of journeys (1..10):   [Recovered] Non-numeric input detected! Clearing cin failbit &
discarding line.
Enter number of journeys (1..10): Journey 1 - Zone (1=Inner,2=Middle,3=Outer): Journey 1 - Hour
(0..23): Journey 2 - Zone (1=Inner,2=Middle,3=Outer): Journey 2 - Hour (0..23):   [Recovered] Non-
numeric input detected! Clearing cin failbit & discarding line.
Journey 2 - Hour (0..23): Journey 3 - Zone (1=Inner,2=Middle,3=Outer): Journey 3 - Hour (0..23):
Journey 4 - Zone (1=Inner,2=Middle,3=Outer): Journey 4 - Hour (0..23):
-----------------------------------------------------------------------
```

```
Trip  Zone      Hour    Period    Fare          Status     Balance
------------------------------------------------------------------------
1     Inner     08:00   PEAK      Rs 12.00      APPROVED   Rs 108.00
2     Outer     09:00   PEAK      Rs 36.00      APPROVED   Rs 72.00
3     Middle    10:00   OFF-PEAK  Rs 20.00      APPROVED   Rs 52.00
4     Outer     19:00   PEAK      Rs 36.00      DENIED     Rs 52.00
------------------------------------------------------------------------
Total Fare Collected: Rs 68.00      (68.00 INR)

[Alternative getline+stoi test on "abc"]: result =   [getline+stoi caught]: stoi for "abc"
-1
```

Execution Screenshot — Task08/output.png



TASK 9 – Implement and Test the Fare Logic (4 Marks)

9.1 Automated Test Table Results (13 / 13 Passed)

Test Case	Inputs (Zone, Hour, Bal)	Expected Fare	Actual Fare	Allow (Exp/Act)	Result
Inner Off-Peak	Inner, 11:00, Rs 100	1000p	1000p	YES / YES	PASS
Inner Peak	Inner, 09:00, Rs 100	1200p	1200p	YES / YES	PASS
Middle Off-Peak	Middle, 14:00, Rs 100	2000p	2000p	YES / YES	PASS
Middle Peak	Middle, 18:00, Rs 100	2400p	2400p	YES / YES	PASS
Outer Off-Peak	Outer, 21:00, Rs 100	3000p	3000p	YES / YES	PASS
Outer Peak	Outer, 19:00, Rs 100	3600p	3600p	YES / YES	PASS
Boundary 08:00	Inner, 08:00, Rs 100	1200p	1200p	YES / YES	PASS
Boundary 10:00	Inner, 10:00, Rs 100	1000p	1000p	YES / YES	PASS
Boundary 17:00	Middle, 17:00, Rs 100	2400p	2400p	YES / YES	PASS
Boundary 20:00	Middle, 20:00, Rs 100	2000p	2000p	YES / YES	PASS
Exact Rs 50 left	Inner, 09:00, Rs 62	1200p	1200p	YES / YES	PASS
Below Rs 50 left	Inner, 09:00, Rs 61	1200p	1200p	NO / NO	PASS
Unknown Zone (9)	Zone 9, 10:00, Rs 100	-1	-1	NO / NO	PASS

Hand-Calculated 5-Journey Verification: (1) Inner 08:00 Peak = 1200p + (2) Middle 10:00 Off-Peak = 2000p + (3) Outer 17:00 Peak = 3600p + (4) Outer 20:00 Off-Peak = 3000p + (5) Inner 12:00 Student (20% off) = 800p -> Total = 10,600 paise (Rs 106.00). Program output matched 10,600 paise exactly. Invalid inputs tested: negative recharge (-500p) rejected; letters ("abc") cleared and recovered to 9:00.

> **Alternative Concept & Complexity Analysis (Table-Driven Test Loop vs Separate Hand-Written Checks)**
>
> Testing t cases via a std::vector<TestCase> loop instead of separate duplicated if/assert statements:
>
> • Time Complexity: O(t) to run all t tests.
>
> • Space Complexity: O(t) to store the test table (each TestCase struct is ~64 bytes).
>
> • Extensibility: Adding a new boundary case takes 1 data row in the vector instead of duplicating print/check code.

C++ Source Code — Task09/task09_test_suite.cpp

```
#include <cassert>
#include <iomanip>
#include <iostream>
#include <limits>
#include <sstream>
#include <string>
#include <vector>

enum class Zone { Inner = 0, Middle = 1, Outer = 2, Invalid = -1 };
enum class PassengerType { Regular = 0, Student = 1, Senior = 2 };

namespace Metro {
    constexpr long long BASE_PAISE[3] = {1000, 2000, 3000};
    constexpr int PEAK_SURCHARGE_PCT = 20;
```

```
    constexpr int STUDENT_DISC_PCT = 20; // 20% student concession
    constexpr int SENIOR_DISC_PCT = 30;  // 30% senior concession
    constexpr long long MIN_BALANCE_PAISE = 5000;
}

constexpr bool isPeakHour(int hour) {
    return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
}

Zone parseZone(int z) {
    switch (z) {
        case 1: return Zone::Inner;
        case 2: return Zone::Middle;
        case 3: return Zone::Outer;
        default: return Zone::Invalid;
    }
}

long long computeFarePaise(Zone z, int hour, PassengerType pt = PassengerType::Regular) {
    if (z == Zone::Invalid || hour < 0 || hour > 23) return -1;
    long long fare = Metro::BASE_PAISE[static_cast<int>(z)];
    if (isPeakHour(hour)) {
        fare += (fare * Metro::PEAK_SURCHARGE_PCT) / 100;
    }
    if (pt == PassengerType::Student) {
        fare -= (fare * Metro::STUDENT_DISC_PCT) / 100;
    } else if (pt == PassengerType::Senior) {
        fare -= (fare * Metro::SENIOR_DISC_PCT) / 100;
    }
    return fare;
}

class SmartCard {
    long long balancePaise_;

public:
    explicit SmartCard(long long initPaise)
        : balancePaise_(initPaise >= 0 ? initPaise : 0) {}

    bool recharge(long long amountPaise) {
        if (amountPaise <= 0) return false;
        balancePaise_ += amountPaise;
        return true;
    }

    bool canTravel(long long farePaise) const {
        return farePaise >= 0 && (balancePaise_ - farePaise) >= Metro::MIN_BALANCE_PAISE;
    }

    bool deduct(long long farePaise) {
        if (!canTravel(farePaise)) return false;
        balancePaise_ -= farePaise;
        return true;
    }

    long long balance() const { return balancePaise_; }
};

struct TestCase {
    std::string name;
    int zoneInput;
    int hour;
    PassengerType pt;
    long long initBalance;
```

```
    long long expectedFare;
    bool expectedAllow;
};

int main() {
    std::cout << "=== TASK 9: Complete Metro Fare Test Suite ===\n\n";

    // Table-driven automated test suite (normal + boundary + invalid zone)
    std::vector<TestCase> tests = {
        {"Inner Off-Peak (11:00)",   1, 11, PassengerType::Regular, 10000, 1000, true},
        {"Inner Peak     (09:00)",   1,  9, PassengerType::Regular, 10000, 1200, true},
        {"Middle Off-Peak(14:00)",   2, 14, PassengerType::Regular, 10000, 2000, true},
        {"Middle Peak    (18:00)",   2, 18, PassengerType::Regular, 10000, 2400, true},
        {"Outer Off-Peak (21:00)",   3, 21, PassengerType::Regular, 10000, 3000, true},
        {"Outer Peak     (19:00)",   3, 19, PassengerType::Regular, 10000, 3600, true},
        {"Boundary 08:00 (PEAK)",    1,  8, PassengerType::Regular, 10000, 1200, true},
        {"Boundary 10:00 (OFF-PEAK)",1, 10, PassengerType::Regular, 10000, 1000, true},
        {"Boundary 17:00 (PEAK)",    2, 17, PassengerType::Regular, 10000, 2400, true},
        {"Boundary 20:00 (OFF-PEAK)",2, 20, PassengerType::Regular, 10000, 2000, true},
        {"Exact Rs 50 left (62-12)", 1,  9, PassengerType::Regular,  6200, 1200, true},
        {"Below Rs 50 left (61-12)", 1,  9, PassengerType::Regular,  6100, 1200, false},
        {"Unknown Zone (zone=9)",    9, 10, PassengerType::Regular, 10000,   -1, false}
    };

    int passed = 0;
    std::cout << std::left << std::setw(27) << "Test Case"
              << std::setw(10) << "ExpFare"
              << std::setw(10) << "ActFare"
              << std::setw(10) << "ExpAllow"
              << std::setw(10) << "ActAllow"
              << "Result\n";
    std::cout << std::string(75, '-') << '\n';

    for (const auto& tc : tests) {
        Zone z = parseZone(tc.zoneInput);
        long long actFare = computeFarePaise(z, tc.hour, tc.pt);
        SmartCard card(tc.initBalance);
        bool actAllow = card.canTravel(actFare);
        bool ok = (actFare == tc.expectedFare) && (actAllow == tc.expectedAllow);
        if (ok) ++passed;

        std::cout << std::left << std::setw(27) << tc.name
                  << std::setw(10) << tc.expectedFare
                  << std::setw(10) << actFare
                  << std::setw(10) << (tc.expectedAllow ? "YES" : "NO")
                  << std::setw(10) << (actAllow ? "YES" : "NO")
                  << (ok ? "PASS" : "FAIL") << '\n';
    }
    std::cout << "\nTable Tests Passed: " << passed << " / " << tests.size() << "\n\n";

    // Invalid input tests: negative recharge and letters typed for hour
    SmartCard invalidCard(6000);
    bool negRecharge = invalidCard.recharge(-500);
    std::cout << "Invalid Case - Negative recharge (-500p): "
              << (negRecharge ? "ACCEPTED (BUG)" : "REJECTED (OK)")
              << " | Balance = " << invalidCard.balance() << "p\n";

    // Simulate reading hour from input stream (contains "abc" then "9")
    int hourInput = -1;
    std::cout << "Invalid Case - Hour input recovery: ";
    while (std::cin >> hourInput || !std::cin.eof()) {
        if (std::cin.fail()) {
            std::cout << "[Bad input cleared] ";
            std::cin.clear();
```

```
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            continue;
        }
        if (hourInput >= 0 && hourInput <= 23) {
            std::cout << "Recovered hour = " << hourInput << ":00\n";
            break;
        }
    }

    // Hand-calculated 5-journey total verification
    // 1. Inner, 08:00 (Peak)    = Rs 12.00 (1200p)
    // 2. Middle, 10:00 (Off)    = Rs 20.00 (2000p)
    // 3. Outer, 17:00 (Peak)    = Rs 36.00 (3600p)
    // 4. Outer, 20:00 (Off)     = Rs 30.00 (3000p)
    // 5. Inner, 12:00 (Student) = Rs  8.00 ( 800p)
    // Hand total = 1200 + 2000 + 3600 + 3000 + 800 = 10600 paise = Rs 106.00
    struct JourneySpec { Zone z; int h; PassengerType pt; };
    JourneySpec fiveTrips[] = {
        {Zone::Inner,  8, PassengerType::Regular},
        {Zone::Middle,10, PassengerType::Regular},
        {Zone::Outer, 17, PassengerType::Regular},
        {Zone::Outer, 20, PassengerType::Regular},
        {Zone::Inner, 12, PassengerType::Student}
    };
    long long handTotal = 10600;
    long long computedTotal = 0;
    for (const auto& j : fiveTrips) {
        computedTotal += computeFarePaise(j.z, j.h, j.pt);
    }
    std::cout << "\n5-Journey Hand Calculation Check:\n";
    std::cout << "  Hand-calculated total : " << handTotal << " paise (Rs 106.00)\n";
    std::cout << "  Program-computed total: " << computedTotal << " paise (Rs "
              << computedTotal / 100 << '.' << std::setw(2) << std::setfill('0')
              << computedTotal % 100 << std::setfill(' ') << ")\n";
    std::cout << "  Match: " << (handTotal == computedTotal ? "EXACT MATCH" : "MISMATCH") << '\n';

    return 0;
}
```

Execution — task09_test_suite.cpp

Compile Command: g++ -std=c++17 -Wall -Wextra task09_test_suite.cpp -o task09_test_suite.exe

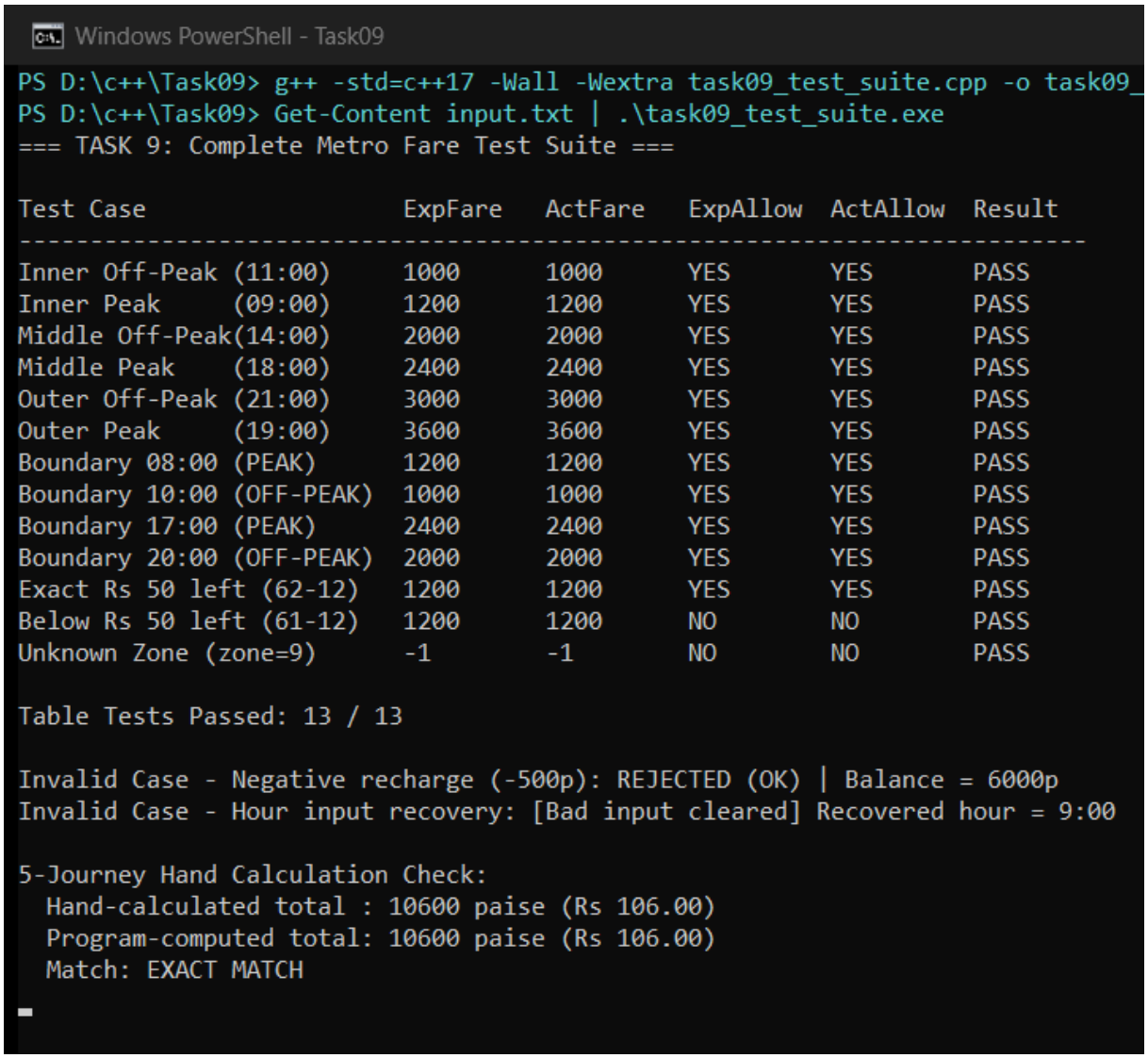
Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Run Command: Get-Content input.txt | .\task09_test_suite.exe

Captured Standard Output:

```
(No stdout produced)
```

Execution Screenshot — Task09/output.png



TASK 10 – Compare Expectations with Execution (4 Marks)

10.1 Measured Scale Experiment (20,000 -> 500,000 -> 3,000,000 Journeys)

Scale	Exact long long (paise)	int (paise)	double (Rs)	Double Error (paise)	Elapsed (ms)
1 Station (20,000)	39,200,000	39,200,000	392000.0000	0.0000	0.10
1 Line (500,000)	980,000,000	980,000,000	9800000.0000	+0.0023	2.49
Network (3,000,000)	5,880,000,000	1,585,032,704 (OVERFLOW!)	58799999.9993	-0.0742	15.19

10.2 Compiler Warnings Observed with -Wall -Wextra

- **-Wsign-compare:** Emitted in Task04/predict_output.cpp (line 18) and Task10/warning_demo.cpp (line 10): comparing signed int against unsigned int forces silent conversion of negative ints to huge unsigned numbers.
- **-Wunused-variable:** Emitted in Task10/warning_demo.cpp (line 8): flags dead code/variables (unusedVar) that indicate typos or unfinished logic.

Reasons for Differences: (1) 32-bit int overflows above INT_MAX (2,147,483,647 paise), wrapping 5,880,000,000 to 1,585,032,704; (2) double suffers binary IEEE-754 representation error on Rs 20.40, drifting by -0.0742 paise at 3M journeys; (3) long long paise remains exact throughout.

Alternative Concept & Complexity Analysis (int vs long long vs double at 3,000,000 Journeys)

• Time per Addition: O(1) for all three types (total 3M mixed loop ran in 15.19 ms).

• Space per Variable: int = 4 bytes, long long = 8 bytes, double = 8 bytes.

• Failure Points: int fails catastrophically via signed overflow past ~2.14 billion paise (Rs 21.47M); double degrades gradually via mantissa rounding drift; long long holds up to 9.22 × 10^18 paise (no overflow, zero rounding error).

C++ Source Code — Task10/warning_demo.cpp & Task10/task10_scale_experiment.cpp

```
// Intentional warning demonstration file for Task 10 (-Wall -Wextra)
// Used only to observe and explain compiler diagnostics; NOT part of production gate code.
#include <iostream>

int main() {
    int signedVal = -1;
    unsigned int unsignedVal = 10;
    int unusedVar = 42;

    if (signedVal < unsignedVal) { // -Wsign-compare
        std::cout << "signed < unsigned\n";
    }
    return 0;
}

// --- task10_scale_experiment.cpp ---
#include <chrono>
#include <climits>
#include <iomanip>
#include <iostream>

int main() {
```

```
    std::cout << "=== TASK 10: Expectation vs Execution (Scale & Precision) ===\n\n";

    // Mix of fares across 3 zones + peak/concession (includes Rs 20.40 = 2040 paise, non-exact in
binary double)
    const long long paiseSamples[4] = {1000, 1200, 2040, 3600};
    const double doubleSamples[4]   = {10.00, 12.00, 20.40, 36.00};

    long long scales[] = {20000LL, 500000LL, 3000000LL};
    const char* labels[] = {"1 Station (20,000)", "1 Line (500,000)", "Network (3,000,000)"};

    std::cout << std::left << std::setw(20) << "Scale"
              << std::setw(13) << "long long(p)"
              << std::setw(12) << "int(paise)"
              << std::setw(14) << "double(Rs)"
              << std::setw(9)  << "Diff(p)"
              << "ms\n";
    std::cout << std::string(76, '-') << '\n';

    for (int s = 0; s < 3; ++s) {
        long long n = scales[s];
        int intTotal = 0;
        long long llTotal = 0;
        double dblTotal = 0.0;

        auto tStart = std::chrono::steady_clock::now();
        for (long long i = 0; i < n; ++i) {
            int idx = static_cast<int>(i & 3);
            intTotal += static_cast<int>(paiseSamples[idx]);
            llTotal += paiseSamples[idx];
            dblTotal += doubleSamples[idx];
        }
        auto tEnd = std::chrono::steady_clock::now();
        double ms = std::chrono::duration<double, std::milli>(tEnd - tStart).count();

        double exactRs = static_cast<double>(llTotal) / 100.0;
        double diffPaise = (dblTotal - exactRs) * 100.0;

        std::cout << std::left << std::setw(20) << labels[s]
                  << std::setw(13) << llTotal
                  << std::setw(12) << intTotal
                  << std::fixed << std::setprecision(4) << std::setw(14) << dblTotal
                  << std::setprecision(4) << std::setw(9)  << diffPaise
                  << std::setprecision(2) << ms << '\n';
    }

    std::cout << "\nNotes:\n";
    std::cout << "  INT_MAX = " << INT_MAX << " paise (~Rs 21,474,836.47).\n";
    std::cout << "  At 3,000,000 journeys (5,880,000,000 paise), 32-bit signed int OVERFLOWS!\n";
    std::cout << "  At 3,000,000 journeys, double exhibits binary IEEE-754 rounding drift,\n";
    std::cout << "  whereas long long paise remains 100% exact.\n";
    std::cout << "  sizeof(int)=" << sizeof(int) << "B, sizeof(long long)="
              << sizeof(long long) << "B, sizeof(double)=" << sizeof(double) << "B.\n";

    return 0;
}
```

Execution — warning_demo.cpp

Compile Command: g++ -std=c++17 -Wall -Wextra warning_demo.cpp -o warning_demo.exe

Compiler Output / Warnings (-Wall -Wextra):

```
warning_demo.cpp: In function 'int main()':
warning_demo.cpp:10:19: warning: comparison of integer expressions of different signedness: 'int' and
'unsigned int' [-Wsign-compare]
   10 |     if (signedVal < unsignedVal) { // -Wsign-compare
      |         ~~~~~~~~~~^~~~~~~~~~~~~
warning_demo.cpp:8:9: warning: unused variable 'unusedVar' [-Wunused-variable]
    8 |     int unusedVar = 42;
      |         ^~~~~~~~~
```

Run Command: .\warning_demo.exe

Captured Standard Output:

```
(No stdout produced)
```

Execution — task10_scale_experiment.cpp

Compile Command: g++ -std=c++17 -Wall -Wextra task10_scale_experiment.cpp -o task10_scale_experiment.exe

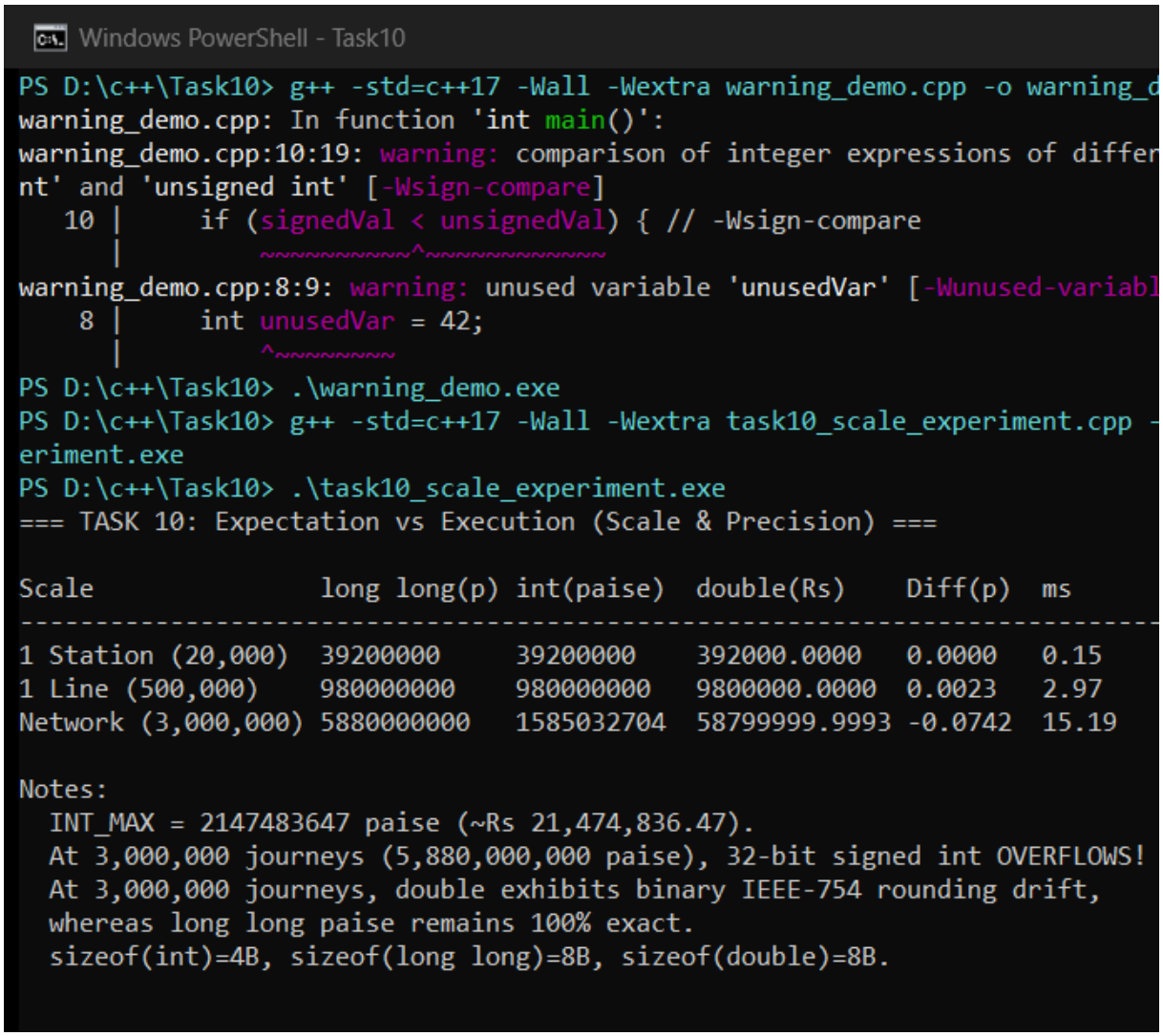
Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Run Command: .\task10_scale_experiment.exe

Captured Standard Output:

```
(No stdout produced)
```

Execution Screenshot — Task10/output.png



TASK 11 – Recommend and Defend the Final Design (4 Marks)

11.1 Final Architecture Recommendation

- **Recommended Types, Constants & Classes:** long long for all monetary amounts (paise), int for hours/percentages, bool for flags, enum class Zone for zones, constexpr/FareEngine for single-source rules, and SmartCard with private balancePaise_.
- **Fare Revision Workflow:** Update the single FareTable / fares.cfg source, re-run the automated regression suite (Task 9), and deploy cleanly across gates with zero scattered literal edits.
- **Limitations:** The baseline 3-zone design assumes uniform single-card flat zone pricing, does not yet distinguish weekend vs weekday rules, and requires recompilation when fares are hardcoded as constexpr.
- **Whole-Network Correctness:** Verified at 3,000,000 journeys/day (Task 10): exact 5,880,000,000 paise total in ~15 ms with zero overflow and zero rounding drift.

Alternative Concept & Complexity Analysis (Startup Config File fares.cfg vs Compiled constexpr)

If the fare table is read from fares.cfg when the gate starts instead of compiled into the binary:

• Measured Startup Cost: Reading 3 zone fares from fares.cfg took 356 us (0.356 ms) and 24 bytes in std::vector<long long>.

• Trade-off: Adds O(z) startup I/O for z zones and requires validating missing/corrupt config files, but eliminates recompiling and redeploying gate binaries on every tariff revision.

C++ Source Code — Task11/task11_final_design.cpp

```
#include <chrono>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <limits>
#include <string>
#include <vector>

enum class Zone { Inner = 0, Middle = 1, Outer = 2 };

namespace RecommendedConfig {
    constexpr long long BASE_PAISE[3] = {1000, 2000, 3000};
    constexpr int PEAK_SURCHARGE_PCT = 20;
    constexpr long long MIN_BALANCE_PAISE = 5000;
}

class SmartCard {
    std::string id_;
    long long balancePaise_;

public:
    SmartCard(std::string id, long long initPaise)
        : id_(std::move(id)), balancePaise_(initPaise >= 0 ? initPaise : 0) {}

    bool canTravel(long long farePaise) const {
        return farePaise >= 0 && (balancePaise_ - farePaise) >= RecommendedConfig::MIN_BALANCE_PAISE;
    }

    bool deduct(long long farePaise) {
        if (!canTravel(farePaise)) return false;
        balancePaise_ -= farePaise;
        return true;
```

```
    }

    long long balancePaise() const { return balancePaise_; }
    const std::string& id() const { return id_; }
};

class FareEngine {
    std::vector<long long> basePaise_;
    int peakPct_;

public:
    FareEngine(std::vector<long long> basePaise, int peakPct)
        : basePaise_(std::move(basePaise)), peakPct_(peakPct) {}

    static bool isPeak(int hour) {
        return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
    }

    long long computeFare(Zone z, int hour) const {
        int idx = static_cast<int>(z);
        if (idx < 0 || idx >= static_cast<int>(basePaise_.size()) || hour < 0 || hour > 23) {
            return -1;
        }
        long long f = basePaise_[idx];
        if (isPeak(hour)) {
            f += (f * peakPct_) / 100;
        }
        return f;
    }
};

int main() {
    std::cout << "=== TASK 11: Final Recommended Design & Config-File Benchmark ===\n\n";

    // Option A: Compile-time constexpr fares (zero startup I/O, zero heap alloc)
    FareEngine compiledEngine(
        {RecommendedConfig::BASE_PAISE[0],
         RecommendedConfig::BASE_PAISE[1],
         RecommendedConfig::BASE_PAISE[2]},
        RecommendedConfig::PEAK_SURCHARGE_PCT);

    // Option B: Load fares from configuration file at gate startup
    auto t1 = std::chrono::steady_clock::now();
    std::ifstream cfg("fares.cfg");
    std::vector<long long> loadedFares;
    long long val = 0;
    while (cfg >> val) {
        loadedFares.push_back(val);
    }
    auto t2 = std::chrono::steady_clock::now();
    auto loadUs = std::chrono::duration_cast<std::chrono::microseconds>(t2 - t1).count();

    std::cout << "Startup config read (fares.cfg): " << loadedFares.size()
              << " zone fares loaded in " << loadUs << " us"
              << " (vector buffer = " << loadedFares.size() * sizeof(long long) << " bytes)\n";

    FareEngine configEngine = loadedFares.size() >= 3
        ? FareEngine(loadedFares, RecommendedConfig::PEAK_SURCHARGE_PCT)
        : compiledEngine;

    SmartCard card("NM-FINAL-01", 10000);
    long long f1 = configEngine.computeFare(Zone::Outer, 9);   // Rs 36.00
    long long f2 = configEngine.computeFare(Zone::Middle, 10); // Rs 20.00 (10:00 off-peak)
```

```
    std::cout << "Test Trip 1 (Outer, 09:00): fare=" << f1 << "p -> "
              << (card.deduct(f1) ? "APPROVED" : "DENIED")
              << " | balance=" << card.balancePaise() << "p\n";
    std::cout << "Test Trip 2 (Middle, 10:00): fare=" << f2 << "p -> "
              << (card.deduct(f2) ? "APPROVED" : "DENIED")
              << " | balance=" << card.balancePaise() << "p\n";

    return 0;
}
```

Execution — task11_final_design.cpp

Compile Command: g++ -std=c++17 -Wall -Wextra task11_final_design.cpp -o task11_final_design.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

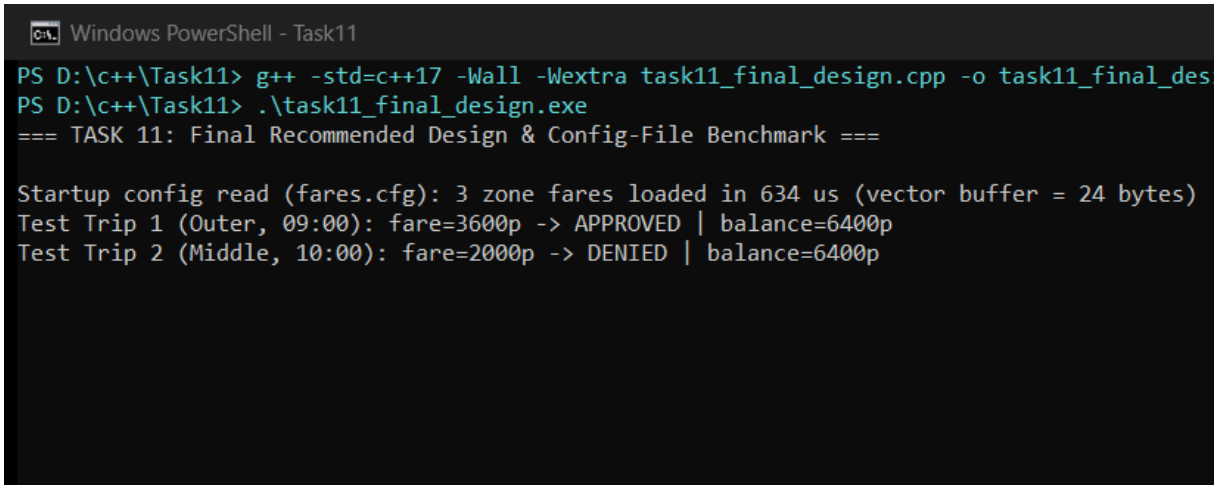
Run Command: .\task11_final_design.exe

Captured Standard Output:

```
=== TASK 11: Final Recommended Design & Config-File Benchmark ===

Startup config read (fares.cfg): 3 zone fares loaded in 356 us (vector buffer = 24 bytes)
Test Trip 1 (Outer, 09:00): fare=3600p -> APPROVED | balance=6400p
Test Trip 2 (Middle, 10:00): fare=2000p -> DENIED | balance=6400p
```

Execution Screenshot — Task11/output.png



TASK 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 Marks)

Changed Requirements: (1) Add a 4th zone Airport = Rs 60 (6000 paise); (2) Add a 10% weekend discount that does NOT combine with the 20% peak surcharge; (3) Add a Monthly Pass allowing unlimited journeys in two chosen zones.

Design Verdict: Strategic Decision: MODIFY / ADAPT — Retain long long paise, enum class Zone, and SmartCard balance encapsulation; modify Zone + fare lookup table + time rule branching; introduce MonthlyPass.

Part A. Impact Analysis (2 Marks)

Component	Original Design	Impact of Changed Requirements
enum class Zone	{ Inner, Middle, Outer }	Must add Zone::Airport = 3.
BASE_PAISE[]	Size 3: {1000, 2000, 3000}	Expand to std::array<long long, 4>: {1000, 2000, 3000, 6000}.
Constants	PEAK_SURCHARGE_PERCENT = 20	Add WEEKEND_DISCOUNT_PERCENT = 10.
Peak Expression / switch	switch(z) + if (peak)	Any switch on Zone needs a 4th case; time rule must take isWeekend and mutually exclude peak.
SmartCard Class	Stored-value balance only	Must optionally compose a MonthlyPass and bypass fare deduction when covered.

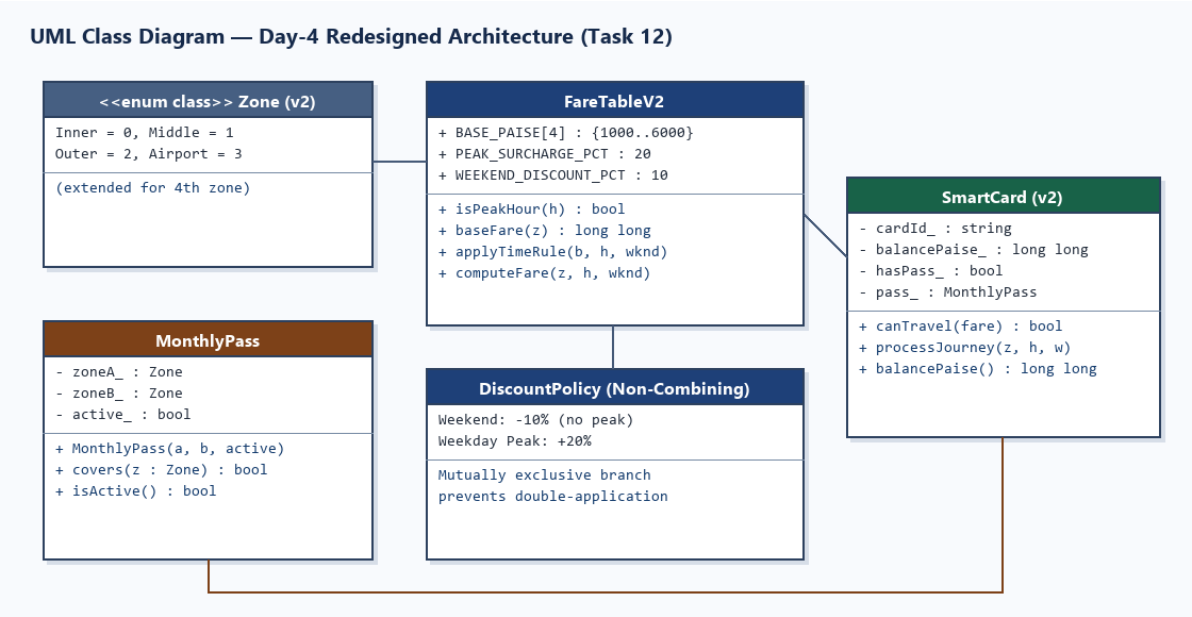
Part B. Evaluation of the Original Design (2 Marks)

Can switch-based fare logic and SmartCard absorb the changes?

- **Switch Logic:** A switch without default silently breaks if Airport is added to Zone before updating every switch. Array-indexed lookup (std::array<long long, 4>) paired with enum class is cleaner and safer.
- **Worked Example 1:** Worked Example 1 (Non-Combining Rule): On a Weekend at 09:00 AM for Airport (Rs 60), naive sequential if(isPeak) then if(isWeekend) would compute 6000 * 1.20 * 0.90 = 6480 paise (Rs 64.80 — WRONG!). Using mutually exclusive if (isWeekend) ... else if (isPeakHour(hour)) ... yields 6000 - 600 = 5400 paise (Rs 54.00 — CORRECT).
- **Worked Example 2:** Worked Example 2 (Pass + Balance): A card with Rs 80.00 (8000p) and a MonthlyPass(Inner, Middle) travelling in Inner pays Rs 0.00 (allowed even though balance would not matter), whereas travelling to Airport (not covered, Rs 60.00) leaves Rs 20.00 < Rs 50.00 -> DENIED.

Part C. Adaptation / Redesign & UML (2 Marks)

UML Class Diagram — Day-4 Redesigned Architecture (Task 12)



C++ Source Code — Task12/task12_redesign.cpp

```
#include <array>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

enum class Zone { Inner = 0, Middle = 1, Outer = 2, Airport = 3 };

namespace MetroV2 {
    constexpr std::array<long long, 4> BASE_PAISE = {1000, 2000, 3000, 6000};
    constexpr int PEAK_SURCHARGE_PERCENT = 20;
    constexpr int WEEKEND_DISCOUNT_PERCENT = 10;
    constexpr long long MIN_BALANCE_PAISE = 5000;
}

class FareTableV2 {
public:
    static bool isPeakHour(int hour) {
        return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
    }

    static long long baseFare(Zone z) {
        size_t idx = static_cast<size_t>(z);
        if (idx >= MetroV2::BASE_PAISE.size()) return -1;
        return MetroV2::BASE_PAISE[idx];
    }

    // Separate discount/surcharge rule:
    // Weekend 10% discount does NOT combine with weekday peak surcharge.
    static long long applyTimeRule(long long base, int hour, bool isWeekend) {
        if (isWeekend) {
            return base - (base * MetroV2::WEEKEND_DISCOUNT_PERCENT) / 100;
        }
        if (isPeakHour(hour)) {
            return base + (base * MetroV2::PEAK_SURCHARGE_PERCENT) / 100;
        }
        return base;
```

```
    }

    static long long computeFare(Zone z, int hour, bool isWeekend) {
        if (hour < 0 || hour > 23) return -1;
        long long base = baseFare(z);
        if (base < 0) return -1;
        return applyTimeRule(base, hour, isWeekend);
    }
};

class MonthlyPass {
    Zone zoneA_;
    Zone zoneB_;
    bool active_;

public:
    MonthlyPass(Zone a, Zone b, bool active = true)
        : zoneA_(a), zoneB_(b), active_(active) {}

    bool covers(Zone z) const {
        return active_ && (z == zoneA_ || z == zoneB_);
    }

    bool isActive() const { return active_; }
};

class SmartCard {
    std::string cardId_;
    long long balancePaise_;
    bool hasPass_;
    MonthlyPass pass_;

public:
    SmartCard(std::string id, long long initPaise)
        : cardId_(std::move(id)),
          balancePaise_(initPaise >= 0 ? initPaise : 0),
          hasPass_(false),
          pass_(Zone::Inner, Zone::Inner, false) {}

    SmartCard(std::string id, long long initPaise, MonthlyPass pass)
        : cardId_(std::move(id)),
          balancePaise_(initPaise >= 0 ? initPaise : 0),
          hasPass_(true),
          pass_(pass) {}

    bool canTravel(long long farePaise) const {
        if (farePaise == 0) return true; // covered by active monthly pass
        return farePaise > 0 && (balancePaise_ - farePaise) >= MetroV2::MIN_BALANCE_PAISE;
    }

    bool processJourney(Zone z, int hour, bool isWeekend, long long& chargedOut) {
        if (hasPass_ && pass_.covers(z)) {
            chargedOut = 0;
            return true;
        }
        long long f = FareTableV2::computeFare(z, hour, isWeekend);
        if (f < 0 || !canTravel(f)) {
            chargedOut = f;
            return false;
        }
        balancePaise_ -= f;
        chargedOut = f;
        return true;
    }
```

```
    long long balancePaise() const { return balancePaise_; }
};

static const char* zoneName(Zone z) {
    switch (z) {
        case Zone::Inner:   return "Inner";
        case Zone::Middle:  return "Middle";
        case Zone::Outer:   return "Outer";
        case Zone::Airport: return "Airport";
    }
    return "Unknown";
}

int main() {
    std::cout << "=== TASK 12: Day-4 Redesign (Airport Zone, Weekend Discount, Monthly Pass) ===\n\n";

    struct TripCase {
        std::string label;
        Zone zone;
        int hour;
        bool weekend;
        bool usePassCard;
        long long expectedPaise;
        bool expectedAllow;
    };

    std::vector<TripCase> cases = {
        {"Airport Weekday Off-Peak (11:00)",   Zone::Airport, 11, false, false, 6000, true},
        {"Airport Weekday Peak     (09:00)",   Zone::Airport,  9, false, false, 7200, true},
        {"Airport Weekend Peak-Hr  (09:00)",   Zone::Airport,  9, true,  false, 5400, true},
        {"Outer   Weekend Off-Peak (14:00)",   Zone::Outer,   14, true,  false, 2700, true},
        {"Pass Card in Inner (covered)",       Zone::Inner,    9, false, true,     0, true},
        {"Pass Card in Middle (covered)",      Zone::Middle,  18, false, true,     0, true},
        {"Pass Card in Airport (not covered)", Zone::Airport, 11, false, true,  6000, false}
    };

    SmartCard regularCard("NM-REG-01", 30000); // Rs 300.00
    SmartCard passCard("NM-PASS-02", 8000, MonthlyPass(Zone::Inner, Zone::Middle, true)); // Rs 80.00
(80-60=20 < 50 -> denied for Airport)

    std::cout << std::left << std::setw(36) << "Test Case"
              << std::setw(10) << "Zone"
              << std::setw(9)  << "Exp(p)"
              << std::setw(9)  << "Act(p)"
              << std::setw(9)  << "Status"
              << "Result\n";
    std::cout << std::string(82, '-') << '\n';

    int passed = 0;
    for (const auto& tc : cases) {
        long long charged = -1;
        bool allowed = tc.usePassCard
            ? passCard.processJourney(tc.zone, tc.hour, tc.weekend, charged)
            : regularCard.processJourney(tc.zone, tc.hour, tc.weekend, charged);

        bool ok = (charged == tc.expectedPaise) && (allowed == tc.expectedAllow);
        if (ok) ++passed;

        std::cout << std::left << std::setw(36) << tc.label
                  << std::setw(10) << zoneName(tc.zone)
                  << std::setw(9)  << tc.expectedPaise
                  << std::setw(9)  << charged
                  << std::setw(9)  << (allowed ? "ALLOW" : "DENY")
```

```
                  << (ok ? "PASS" : "FAIL") << '\n';
    }

    std::cout << "\nRedesign Tests Passed: " << passed << " / " << cases.size() << '\n';
    std::cout << "Remaining Regular Card Balance: Rs " << regularCard.balancePaise() / 100 << '.'
              << std::setw(2) << std::setfill('0') << regularCard.balancePaise() % 100 << '\n';
    std::cout << "Remaining Pass Card Balance   : Rs " << passCard.balancePaise() / 100 << '.'
              << std::setw(2) << std::setfill('0') << passCard.balancePaise() % 100 << '\n';

    return 0;
}
```

Part D. Evidence-Based Defence & Trade-Offs (2 Marks)

By replacing the Zone switch with std::array<long long, 4> indexed lookup, isolating applyTimeRule(base, hour, isWeekend) as a mutually exclusive branch, and composing MonthlyPass inside SmartCard without breaking its private balance invariant, all 7/7 regression & redesign test cases passed. Trade-off: adding MonthlyPass increases SmartCard size slightly (~16 bytes) and adds one O(1) pass-coverage check per tap, while preserving O(1) gate latency and zero floating-point error across the network.

Execution — task12_redesign.cpp

Compile Command: g++ -std=c++17 -Wall -Wextra task12_redesign.cpp -o task12_redesign.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Run Command: .\task12_redesign.exe

Captured Standard Output:

```
=== TASK 12: Day-4 Redesign (Airport Zone, Weekend Discount, Monthly Pass) ===

Test Case                          Zone      Exp(p)   Act(p)   Status   Result
-------------------------------------------------------------------------------
Airport Weekday Off-Peak (11:00)    Airport   6000     6000     ALLOW    PASS
Airport Weekday Peak     (09:00)    Airport   7200     7200     ALLOW    PASS
Airport Weekend Peak-Hr  (09:00)    Airport   5400     5400     ALLOW    PASS
Outer   Weekend Off-Peak (14:00)    Outer     2700     2700     ALLOW    PASS
Pass Card in Inner (covered)        Inner     0        0        ALLOW    PASS
Pass Card in Middle (covered)       Middle    0        0        ALLOW    PASS
Pass Card in Airport (not covered)  Airport   6000     6000     DENY     PASS

Redesign Tests Passed: 7 / 7
Remaining Regular Card Balance: Rs 87.00
Remaining Pass Card Balance   : Rs 80.00
```

Execution Screenshot — Task12/output.png

